

# Assignment 1 DMI

Montserrat Palazón Balmaseda (montserrat.palazon01@upf.edu)

Last update: 27 marzo, 2026

## Contents

|                        |           |
|------------------------|-----------|
| <b>Exercise 1</b>      | <b>1</b>  |
| Exercise 1.1 . . . . . | 1         |
| Exercise 1.2 . . . . . | 6         |
| <b>Exercise 2</b>      | <b>19</b> |
| Exercise 2.1 . . . . . | 19        |
| Exercise 2.2 . . . . . | 27        |
| <b>Exercise 3</b>      | <b>34</b> |
| <b>Conclusion</b>      | <b>44</b> |
| <b>session info</b>    | <b>44</b> |

This practical focuses on data wrangling, exploratory data analysis, visualization, and critical comparison with published results using real biomedical data.

We will work with the publication **Proteomic and Metabolomic Characterization of COVID-19 Patient Sera**

## Exercise 1

Download and load Supplementary Table 1 (“Additional Demographical and Baseline Characteristics of COVID-19 Patients and Control Groups”) from the supplementary materials of the publication.

### Exercise 1.1

Reproduce Table 1 from the manuscript using the supplementary data. Match the reported summary statistics as closely as possible.

```
# =====  
# Exercise 1.1 - Reproduce Table 1  
# =====
```

```

# ---- 1. Libraries ----
# Loaded in the setup

# ---- 2. Load data ----
data <- read_excel(
  "data/raw_data/1-s2.0-S0092867420306279-mmc1.xlsx",
  sheet = "Clinical_information"
)

# ---- 3. Cleaning and Re-coding ----
data_table_sup1 <- data %>%
  mutate(
    # Main groups
    covid_status = case_when(
      `Group d` == 0 ~ "Healthy Control",
      `Group d` == 1 ~ "Non-COVID-19",
      `Group d` == 2 ~ "Non-severe",
      `Group d` == 3 ~ "Severe"
    ),

    # Cleaning
    # We detected that 'NAs' in 'date of progression to severe state' are indicated as '/'
    # Replace '/' with NA
    severe_date_raw = na_if(`Date of progression to severe state`, "/"),
    severe_date = as.Date(as.numeric(severe_date_raw), origin = "1899-12-30"),

    # Sex
    Sex = factor(`Sex g`,
      levels = c(1, 0),
      labels = c("Male", "Female")),

    # Numeric variables
    Age = `Age (year)`,
    # The BMI variable is as a 'chr' so, we make it a numeric variable
    BMI = as.numeric(na_if(`BMI h`, "/")),

    # Time from onset to admission in days
    # Compute the difference between dates
    onset_to_admission = as.numeric(difftime(`Admission date`, `Onset date f`, units = "days")),

    # Time from admission to severe in days (only for severe patients)
    admission_to_severe = case_when(
      covid_status != "Severe" ~ NA_real_,
      is.na(severe_date) ~ NA_real_,
      severe_date < `Admission date` ~ NA_real_,
      TRUE ~ as.numeric(difftime(severe_date, `Admission date`, units = "days"))
    )
  )

# ---- 4. Table 1: Main groups ----
table_main <- data_table_sup1 %>%
  filter(covid_status %in% c("Healthy Control", "Non-COVID-19")) %>%

```

```

select(covid_status, Sex, Age, BMI) %>%
tbl_summary(
  by = covid_status,
  type = all_continuous() ~ "continuous2",
  statistic = list(
    all_continuous() ~ c(
      "{mean} ± {sd}",
      "{median} ({p25}-{p75})",
      "{min}-{max}"
    ),
    all_categorical() ~ "{n} ({p}%)",
  ),
  digits = all_continuous() ~ 1,
  missing = "no"
)

# ---- 5. Table COVID-19: severity ----
table_covid_nonsev <- data_table_sup1 %>%
  filter(covid_status %in% c("Non-severe")) %>%
  select(covid_status, Sex, Age, BMI, onset_to_admission) %>%
  tbl_summary(
    by = covid_status,
    type = all_continuous() ~ "continuous2",
    statistic = list(
      all_continuous() ~ c(
        "{mean} ± {sd}",
        "{median} ({p25}-{p75})",
        "{min}-{max}"
      ),
      all_categorical() ~ "{n} ({p}%)",
    ),
    digits = all_continuous() ~ 1,
    missing = "no"
  )

table_covid_sev <- data_table_sup1 %>%
  filter(covid_status %in% c("Severe")) %>%
  select(covid_status, Sex, Age, BMI, onset_to_admission, admission_to_severe) %>%
  tbl_summary(
    by = covid_status,
    type = list(
      admission_to_severe ~ "continuous2",
      all_continuous() ~ "continuous2"
    ),
    statistic = list(
      all_continuous() ~ c(
        "{mean} ± {sd}",
        "{median} ({p25}-{p75})",
        "{min}-{max}"
      ),
      all_categorical() ~ "{n} ({p}%)",
    ),
    digits = all_continuous() ~ 1,
    missing = "no"
  )

```

```

    ),
    digits = all_continuous() ~ 1,
    missing = "no"
  )

# ---- 6. Table Total COVID-19 ----
table_covid_total <- data_table_sup1 %>%
  filter(covid_status %in% c("Non-severe", "Severe")) %>%
  select(Sex, Age, BMI, onset_to_admission) %>%
  tbl_summary(
    type = all_continuous() ~ "continuous2",
    statistic = list(
      all_continuous() ~ c(
        "{mean} ± {sd}",
        "{median} ({p25}-{p75})",
        "{min}-{max}"
      ),
      all_categorical() ~ "{n} ({p}%)",
    ),
    digits = all_continuous() ~ 1,
    missing = "no"
  )

# ---- 7. Merge tables ----
table1 <- tbl_merge(
  tbls = list(table_main, table_covid_total, table_covid_nonsev, table_covid_sev),
  tab_spanner = c(
    NA,
    "COVID-19",
    "COVID-19",
    "COVID-19"
  ),
  quiet = TRUE
)

# ---- 8. Final formatting ----
table1_gt <- table1 %>%
  modify_header(label ~ "***Variables**", stat_0_2 ~ "***Total**<br>N = {N}") %>%
  modify_table_body(
    ~ .x %>%
    mutate(
      label = case_when(
        row_type == "label" & label == "Sex" ~ "Sex - no. (%)",
        row_type == "label" & label == "Age" ~ "Age - year",
        row_type == "label" & label == "BMI" ~ "BMI, kg/m²",
        row_type == "label" & label == "onset_to_admission" ~
          "Time from Onset to Admission, Days",
        row_type == "label" & label == "admission_to_severe" ~
          "Time from Admission to Severe, Days",

        label == "Median (Q1-Q3)" ~ "Median (IQR)",
        label == "Min-Max" ~ "Range",
        TRUE ~ label
      )
    )
  )

```

Table 1. Demographics and Baseline Characteristics of COVID-19 Patients

| Variables                           | Healthy Control N = 28 | Non-COVID-19 N = 25 | Total<br>N = 65 |
|-------------------------------------|------------------------|---------------------|-----------------|
| Sex - no. (%)                       |                        |                     |                 |
| Male                                | 21 (75%)               | 17 (68%)            |                 |
| Female                              | 7 (25%)                | 8 (32%)             |                 |
| Age - year                          |                        |                     |                 |
| Mean $\pm$ SD                       | 44.4 $\pm$ 8.3         | 49.2 $\pm$ 14.0     |                 |
| Median (IQR)                        | 45.0 (38.0-51.0)       | 53.0 (38.0-61.0)    | 47              |
| Range                               | 28.0-57.0              | 23.0-67.0           |                 |
| BMI, kg/m <sup>2</sup>              |                        |                     |                 |
| Mean $\pm$ SD                       | 24.4 $\pm$ 2.7         | 23.5 $\pm$ 2.7      |                 |
| Median (IQR)                        | 24.2 (22.5-26.3)       | 24.7 (21.0-25.6)    | 24              |
| Range                               | 19.9-32.9              | 19.1-27.4           |                 |
| Time from Onset to Admission, Days  |                        |                     |                 |
| Mean $\pm$ SD                       |                        |                     |                 |
| Median (IQR)                        |                        |                     | 3               |
| Range                               |                        |                     |                 |
| Time from Admission to Severe, Days |                        |                     |                 |
| Mean $\pm$ SD                       |                        |                     |                 |
| Median (IQR)                        |                        |                     |                 |
| Range                               |                        |                     |                 |

```

    )
  )
) %>%
modify_footnote(
  all_stat_cols() ~ NA
) %>%
as_gt() %>%
tab_options(
  table.font.size = px(8)
) %>%
tab_header(
  title = "Table 1. Demographics and Baseline Characteristics of COVID-19 Patients"
) %>%
opt_all_caps() %>%
tab_style(
  style = cell_fill(color = "white"),
  locations = cells_body(
    rows = row_type == "label"
  )
) %>%
tab_style(
  style = cell_fill(color = "#F2F2F2"),
  locations = cells_body(
    rows = row_type %in% c("level", "stat")
  )
)
)

# Show tabla
table1_gt

```

```

# ---- 8. Save as HTML ----
gtsave(table1_gt, "data/processed_data/Table1_reproduced.html")

```

Briefly discuss any discrepancies between your results and the published table, and provide possible explanations.

Some discrepancies are observed between the reproduced table and the published one. First, several variables present in the original article (such as comorbidities, symptoms, and oxygenation index) are not included in our reproduced table, as these variables are not available in the provided dataset.

For the variables that are available, the summary statistics are very similar, although some differences can be observed, particularly within the COVID-19 groups. While the values for Healthy Control and Non-COVID-19 groups match closely, slight discrepancies appear in the Non-severe and Severe groups.

A possible explanation for these differences could be related to variations in how subgroup counts were computed or reported (male and females counts don't match).

## Exercise 1.2

Perform an exploratory data analysis (EDA) of the dataset. At a minimum, include:

- Descriptive statistics for key variables
- Appropriate visualizations (e.g. distributions, group comparisons)
- A brief written summary of notable patterns, anomalies, or data quality issues

## Cleaning

```
# Distinguish attributes
# We executed str(data) in order to know the structure of the data and the type of each attribute
str(data)

## tibble [118 x 31] (S3: tbl_df/tbl/data.frame)
##  $ Patient ID a          : chr [1:118] "XG1" "XG2" "XG3" "XG4" ...
##  $ Proteomics            : chr [1:118] "Yes" "Yes" "Yes" "Yes" ...
##  $ MS ID b              : chr [1:118] "F1_126" "F1_127N" "F1_127C" "F2_126" ...
##  $ MSRep ID c           : chr [1:118] "F1_133N" "/" "/" "/" ...
##  $ Group d              : num [1:118] 2 2 2 2 2 2 2 2 2 2 ...
##  $ COVID-19 patients corhort : chr [1:118] "Training (C1)" "Training (C1)" "Training (C1)" ...
##  $ Metabolomics analysis : chr [1:118] "Yes" "Yes" "Yes" "Yes" ...
##  $ Metabolomics ID e     : chr [1:118] "PT1" "PT2" "PT3" "PT4" ...
##  $ Onset date f          : POSIXct[1:118], format: "2020-01-24" "2020-01-31" ...
##  $ Admission date        : POSIXct[1:118], format: "2020-01-25" "2020-02-02" ...
##  $ Date of progression to severe state: chr [1:118] "/" "/" "/" "/" ...
##  $ Sampling date for metabolomics : POSIXct[1:118], format: "2020-01-26" "2020-02-03" ...
##  $ Sampling date for proteomics   : chr [1:118] "43866" "43873" "43866" "43863" ...
##  $ Discharge Date          : POSIXct[1:118], format: "2020-02-14" "2020-02-12" ...
##  $ Sex g                  : num [1:118] 1 1 1 1 1 0 0 1 1 1 ...
##  $ Age (year)              : num [1:118] 38 36 70 33 33 45 46 21 63 27 ...
##  $ BMI h                  : chr [1:118] "28.01022373" "24.91349481" "25.88057064" "27.04" ...
##  $ Test date...18         : POSIXct[1:118], format: "2020-01-26" "2020-02-02" ...
##  $ WBC count, ×109/L      : num [1:118] 11.7 4.93 3.43 3.76 3.17 5.09 5.24 7.14 3.86 6.5 ...
##  $ Lymphocyte count, ×109/L : num [1:118] 2.679 1.642 0.722 1.459 1.1 ...
##  $ Monocyte count, ×109/L  : num [1:118] 1.24 0.498 0.31 0.222 0.4 ...
##  $ Platelet count, ×109/L  : num [1:118] 181 194 205 144 144 221 205 211 145 199 ...
##  $ Test date...23         : POSIXct[1:118], format: "2020-01-26" "2020-02-02" ...
```

```
## $ CRP i, mg/L : chr [1:118] "17.9" "1.9" "89.9" "0.5" ...
## $ ALT j, U/L : num [1:118] 52 48.2 22 27.1 30 10 16 37.3 20 16.2 ...
## $ AST k, U/L : num [1:118] 37 34 30 23 30 19 19 21 28 18 ...
## $ GGT l, U/L : num [1:118] 55 59 30 23 29 13 21 48 16 20 ...
## $ TBIL m, mol/L : num [1:118] 8.2 14.1 12.3 18.9 13.1 6 5.9 14.7 15.3 23 ...
## $ DBIL n, mol/L : num [1:118] 2.5 4.8 3.9 6.3 5.5 2.5 2.1 5.7 6.4 8.5 ...
## $ Creatinine, mol/L : num [1:118] 80 94 93 88 110 61 55 88 84 77 ...
## $ Glucose, mmol/L : num [1:118] 6.18 5.25 9.21 5.88 5.12 4.61 7.21 8.12 5.26 6.1
```

# ``` # CLEANING ```

```
data_cleaned <- data
```

```
# New names for better manipulation of the data
```

```
# Previously, very long and complex names (with spaces, units, etc)
```

```
new_names <- c("patient_id", "proteomics", "ms_id", "msrep_id", "group", "covid_cohort", "metabolomics", "metabo")
names(data_cleaned) <- new_names
```

```
# Cleaning incorrect formatting variables
```

```
# We detected that 'NAs' in 'date of progression to severe state' are indicated as '/'
```

```
# Replace '/' with NA in several variable with this incorrect format
```

```
data_cleaned$severe_date <- na_if(data_cleaned$severe_date, "/")
data_cleaned$ms_id <- na_if(data_cleaned$ms_id, "/")
data_cleaned$msrep_id <- na_if(data_cleaned$msrep_id, "/")
data_cleaned$covid_cohort <- na_if(data_cleaned$covid_cohort, "/")
data_cleaned$sampling_date_proteomics <- na_if(data_cleaned$sampling_date_proteomics, "/")
data_cleaned$bmi <- as.numeric(na_if(data_cleaned$bmi, "/"))
data_cleaned$`crp_mg/L` <- as.numeric(na_if(data_cleaned$`crp_mg/L`, "/"))
data_cleaned$metabolomics_id <- na_if(data_cleaned$metabolomics_id, "/")
```

```
# Reformat correctly the date of severe state
```

```
data_cleaned$severe_date <- as.Date(as.numeric(data_cleaned$severe_date), origin = "1899-12-30")
```

```
# Convert BMI and CRP to numeric
```

```
data_cleaned$bmi <- as.numeric(data_cleaned$bmi)
data_cleaned$`crp_mg/L` <- as.numeric(data_cleaned$`crp_mg/L`)
```

```
# Add variable:
```

```
# time from onset to admission -> compute the difference between dates
```

```
data_cleaned$onset_to_admission = as.numeric(difftime(data_cleaned$admission_date, data_cleaned$onset_date))
```

```
# time from admission to severe
```

```
data_cleaned$admission_to_severe = case_when(
  is.na(data_cleaned$severe_date) ~ NA_real_,
  data_cleaned$severe_date < data_cleaned$admission_date ~ NA_real_,
  TRUE ~ as.numeric(difftime(data_cleaned$severe_date, data_cleaned$admission_date, units = "days"))
)
```

```
# RENAME the categorical variables
```

```
# Rename the value of the 'group' variable to uncode them to make them more understandable
```

```
data_cleaned$group <- case_when(
  data_cleaned$group == 0 ~ "Healthy Control",
  data_cleaned$group == 1 ~ "Non-COVID-19",
  data_cleaned$group == 2 ~ "Non-severe",
  data_cleaned$group == 3 ~ "Severe"
)
```

```

)
# Rename the value of the 'sex' variable to make it more understandable
data_cleaned$sex <- case_when(
  data_cleaned$sex == 1 ~ "Male",
  data_cleaned$sex == 0 ~ "Female"
)

```

## Checking data balancing

```

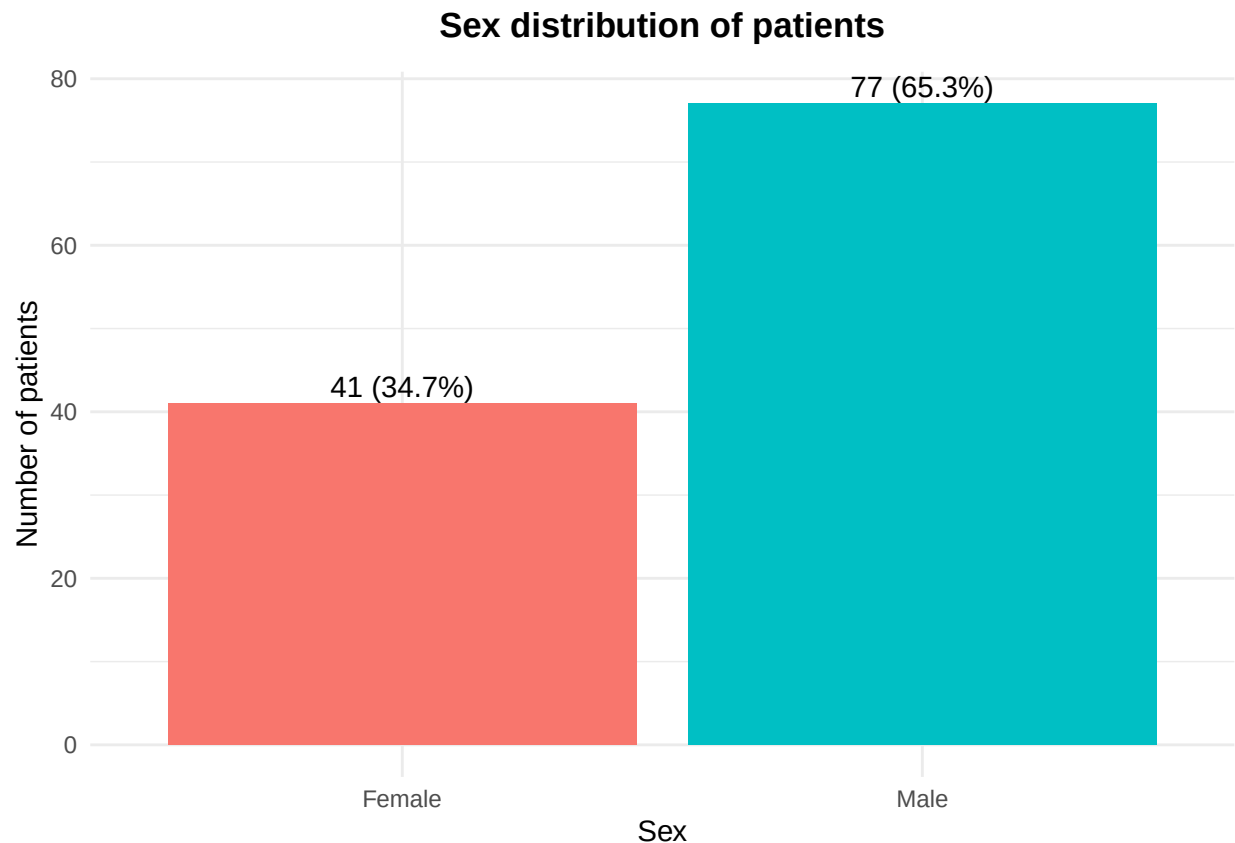
# Distribution of sex
sex_dist <- data_cleaned %>%
  filter(!is.na(sex)) %>%
  count(sex) %>%
  mutate(percentage = n / sum(n) * 100)

# Bar plot
p_sex <- ggplot(sex_dist, aes(x = sex, y = n, fill = sex)) +
  geom_bar(stat = "identity") +
  geom_text(aes(label = paste0(n, " (", round(percentage, 1), "%)")),
    vjust = -0.3, size = 4) +
  labs(
    title = "Sex distribution of patients",
    x = "Sex",
    y = "Number of patients"
  ) +
  theme_minimal() +
  theme(legend.position = "none",
    plot.title = element_text(hjust = 0.5, face = "bold", margin = margin(b = 10)))

p_sex

```





```
ggsave(  
  filename = "data/processed_data/sex_distribution.png",  
  plot = p_sex,  
  width = 6,  
  height = 4,  
  dpi = 300  
)
```

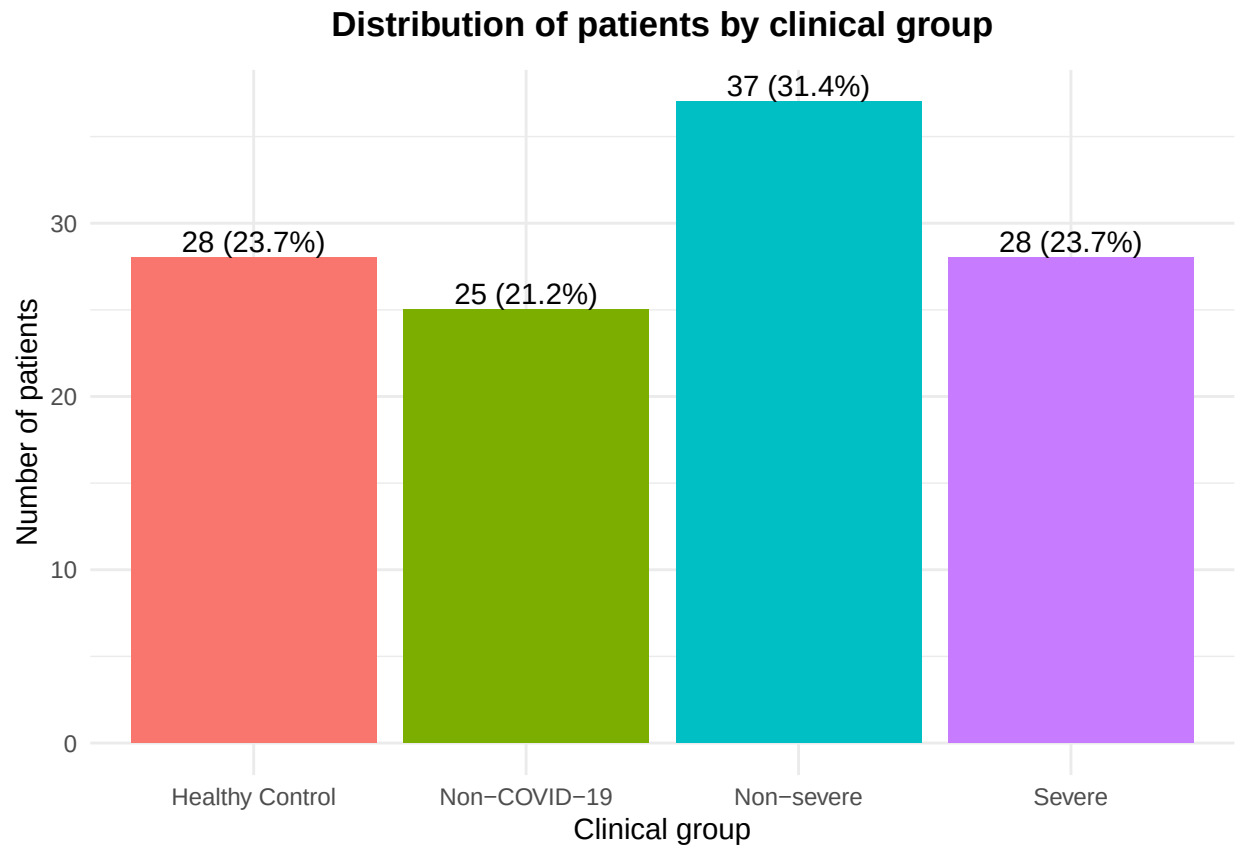
In terms of sex, the dataset is unbalanced, with 65% of the patients being male, whereas we would expect an approximately equal distribution of 50% for each sex.

```
# Distribution of clinical groups  
group_dist <- data_cleaned %>%  
  filter(!is.na(group)) %>%  
  count(group) %>%  
  mutate(  
    percentage = n / sum(n) * 100  
  )  
  
# Bar plot  
p_groups <- ggplot(group_dist, aes(x = group, y = n, fill = group)) +  
  geom_bar(stat = "identity") +  
  geom_text(aes(label = paste0(n, " (", round(percentage, 1), "%)"),  
    vjust = -0.3, size = 4) +  
  labs(  
    title = "Distribution of patients by clinical group",
```

```

x = "Clinical group",
y = "Number of patients"
) +
theme_minimal() +
theme(legend.position = "none",
      plot.title = element_text(hjust = 0.5, face = "bold", margin = margin(b = 10)))
p_groups

```



```

ggsave(
  filename = "data/processed_data/clinicalgroup_distribution.png",
  plot = p_groups,
  width = 6,
  height = 4,
  dpi = 300
)

```

Regarding the 'group' variable, which reflects patient severity, the categories are generally well balanced, except for the 'non-severe' group, which accounts for 31% of the patients, while the other groups represent between 21% and 23% each.

Despite this results, we think that there is no need to do balancing.

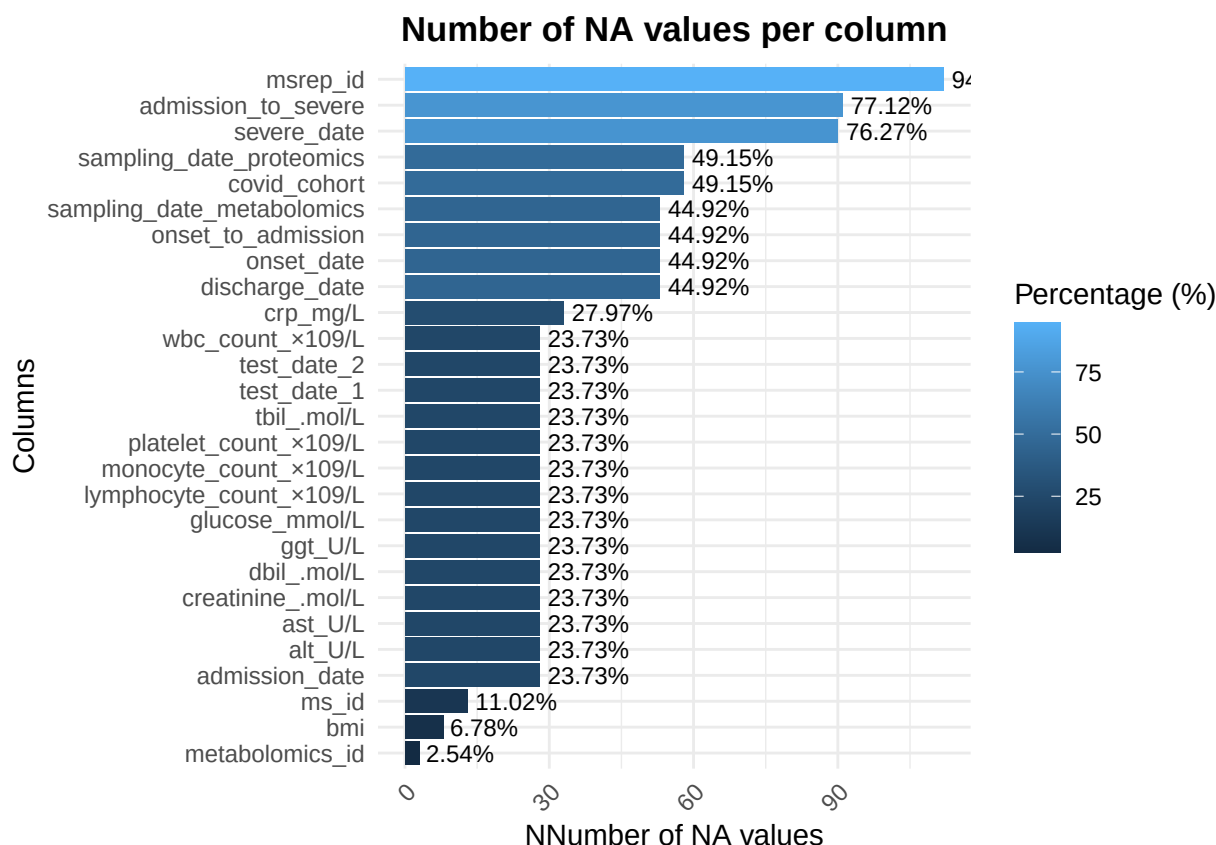
## Detection of Missing data (NA)

```
# First, check for missing values (NA) in the dataset
# Compute NA count and percentage per column
na_summary <- data.frame(column = names(data_cleaned), na_count = colSums(is.na(data_cleaned)), na_perce

# Show only columns with NA values sorted by na_count
na_summary <- na_summary %>% filter(na_count > 0) %>% arrange(na_count)

# Bar plot of NA counts per column
p_nas <- ggplot(na_summary, aes(x = reorder(column, na_count), y = na_count, fill = na_percentage)) +
  geom_bar(stat = "identity") +
  geom_text(aes(label = paste0(round(na_percentage, 2), "%")),
            hjust = -0.1, size = 3) + # Adjust position of text
  coord_flip() + # To make the bars horizontal
  labs(
    title = "Number of NA values per column",
    x = "Columns",
    y = "NNumber of NA values",
    fill = "Percentage (%)"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold"),
    axis.text.x = element_text(angle = 45, hjust = 1)
  )

p_nas
```



```
ggsave(
  filename = "data/processed_data/nas_distribution.png",
  plot = p_nas,
  width = 6,
  height = 4,
  dpi = 300
)
```

As shown in the plot, the technical replicate sample ID is missing in approximately 94% of the patients, which likely indicates that technical replicates were not performed for most of the individuals. However, these IDs are not relevant for our analysis, as they do not provide information related to disease severity (the main variable of interest) and therefore they will not be considered.

The variables with the second highest proportion of missing values are the date of progression to severe disease and, consequently, the number of days between admission and progression to severe status. Since these variables are only applicable to patients who developed severe disease the % of missing data makes sense. We will exclude the corresponding data variables while retaining the derived numerical variables that represent the time intervals between disease states. The same will be applied to the onset date and the time from onset to admission.

It is important to note that although several variables present missing values for up to 76% of the patients, this is expected, as these missing values correspond to patients who never progressed to a severe state. Therefore, no imputation is performed for these variables. Additionally, one extra missing value is observed due to a data entry error in which the recorded date of progression to severe disease precedes the admission date.

The next group of variables with a high proportion of missing values includes those collected exclusively for COVID-19 patients (both non-severe and severe), resulting in approximately 44–49% missings. These

variables mainly correspond to dates, and as we previously commented there are not directly relevant for the current analysis, so we retain only the derived time intervals, such as the number of days between onset and admission.

Furthermore, biological measurements were obtained only for COVID-19 patients, therefore, these variables are missing for approximately 24% of the dataset, corresponding to healthy control individuals so, no imputation is applied. Similarly, the proteomics and metabolomics sample IDs are missing for 11% and 2.5% of patients, respectively and since these identifiers are not informative for severity analysis, they are not imputed.

Finally, body mass index (BMI) is missing for approximately 7% of patients. As there is no clear biological or technical explanation for this missings, it is likely due to incomplete data recording. Therefore, BMI values will be imputed within groups to preserve potential group-specific patterns. The same will be applied to crp which has 5 random NAs.

### Imputation of the NAs of BMI

```
# At data_cleaned we will maintain only the variables that we will use for the analysis
# Remove the following columns
data_cleaned <- data_cleaned %>%
  select(-c(proteomics, ms_id, msrep_id, covid_cohort, metabolomics, metabolomics_id, onset_date, admis

# IMPUTATION of NAs in BMI and CRP
# We want to impute the missing values but by groups
# So, for instance, to impute the missing values of patients of the group 0 only BMIs of the patients w

# Check that the number of NAs in BMI and CRP
sum(is.na(data_cleaned$bmi))
```

```
## [1] 8
```

```
sum(is.na(data_cleaned$`crp_mg/L`))
```

```
## [1] 33
```

```
data_cleaned <- data_cleaned %>%
  group_by(group, sex) %>%
  mutate(
    bmi = ifelse(
      is.na(bmi),
      mean(bmi, na.rm = TRUE),
      bmi
    ),
    `crp_mg/L` = ifelse(
      is.na(`crp_mg/L`) & group %in% c("Non-COVID-19", "Non-severe", "Severe"),
      median(`crp_mg/L`, na.rm = TRUE),
      `crp_mg/L`
    )
  ) %>%
  ungroup()

# Check that there are no more NAs in BMI
sum(is.na(data_cleaned$bmi))
```

```
## [1] 0
```

```
sum(is.na(data_cleaned$`crp_mg/L`))
```

```
## [1] 28
```

## Descriptive statistics for the key variables

```
# As we have cleaned the data, now we can perform the EDA with the remaining variables that are the key  
# First, we will separate categorical and numeric variables for better analysis  
# -> Select only categorical variables  
# We don't select the Patient ID so, as categorical we only have 'sex' and 'group'  
categorical_cols <- c("sex", "group")  
  
# -> Select only numeric variables  
numeric_cols <- c("age", "bmi", "wbc_count_×109/L", "lymphocyte_count_×109/L",  
                  "monocyte_count_×109/L", "platelet_count_×109/L",  
                  "crp_mg/L", "alt_U/L", "ast_U/L", "ggt_U/L", "tbil_mol/L",  
                  "dbil_mol/L", "creatinine_mol/L", "glucose_mmol/L",  
                  "onset_to_admission", "admission_to_severe")  
  
data_numeric <- data_cleaned %>%  
  select(all_of(numeric_cols))  
  
data_categorical <- data_cleaned %>%  
  select(all_of(categorical_cols))  
  
# Univariate Analysis  
# - Numeric variables: descriptive statistics  
# Despite we have already done descriptive statistics in Exercise 1.1, now we will do it again but for  
  
numeric_summary <- data_numeric %>%  
  summarise(  
    across(  
      everything(),  
      list(  
        n = ~sum(!is.na(.)),  
        mean = ~mean(., na.rm = TRUE),  
        sd = ~sd(., na.rm = TRUE),  
        median = ~median(., na.rm = TRUE),  
        min = ~min(., na.rm = TRUE),  
        max = ~max(., na.rm = TRUE)  
      ),  
      .names = "{.col}_{.fn}"  
    )  
  ) %>%  
  pivot_longer(  
    cols = everything(),  
    names_to = c("Variable", "Statistic"),  
    names_sep = "_(?=[^_]+$)"  
  ) %>%
```

```

pivot_wider(
  names_from = Statistic,
  values_from = value
)

numeric_summary %>%
  mutate(across(where(is.numeric), ~round(., 2))) %>%
  kable(
    caption = "Descriptive statistics of numeric variables",
    align = "lrrrrrr",
    col.names = c("Variable", "n", "Mean", "SD", "Median", "Min", "Max")
  ) %>%
  kable_styling(
    bootstrap_options = c("striped", "hover", "condensed"),
    full_width = FALSE,
    position = "center"
  )

```

Table 1: Descriptive statistics of numeric variables

| Variable                | n   | Mean   | SD    | Median | Min   | Max    |
|-------------------------|-----|--------|-------|--------|-------|--------|
| age                     | 118 | 47.42  | 12.82 | 47.00  | 18.00 | 77.00  |
| bmi                     | 118 | 24.50  | 2.79  | 24.39  | 18.94 | 32.87  |
| wbc_count_×109/L        | 90  | 6.92   | 3.55  | 6.11   | 1.90  | 24.20  |
| lymphocyte_count_×109/L | 90  | 1.26   | 0.69  | 1.18   | 0.20  | 4.10   |
| monocyte_count_×109/L   | 90  | 0.48   | 0.20  | 0.45   | 0.10  | 1.24   |
| platelet_count_×109/L   | 90  | 195.61 | 57.94 | 199.50 | 37.00 | 334.00 |
| crp_mg/L                | 90  | 32.15  | 51.31 | 15.00  | 0.10  | 299.70 |
| alt_U/L                 | 90  | 38.62  | 86.17 | 23.55  | 7.00  | 824.00 |
| ast_U/L                 | 90  | 41.86  | 80.07 | 26.00  | 10.00 | 743.00 |
| ggt_U/L                 | 90  | 39.59  | 36.82 | 28.50  | 9.00  | 264.00 |
| tbil_mol/L              | 90  | 20.31  | 43.76 | 13.40  | 3.40  | 403.40 |
| dbil_mol/L              | 90  | 9.02   | 27.46 | 4.75   | 0.80  | 254.40 |
| creatinine_mol/L        | 90  | 82.26  | 35.71 | 76.50  | 4.00  | 335.00 |
| glucose_mmol/L          | 90  | 7.48   | 3.65  | 6.26   | 3.65  | 25.63  |
| onset_to_admission      | 65  | 5.00   | 4.42  | 3.00   | 0.00  | 23.00  |
| admission_to_severe     | 27  | 1.48   | 1.63  | 1.00   | 0.00  | 6.00   |

```

# Distribution plots
data_numeric_long <- data_numeric %>%
  pivot_longer(
    cols = everything(),
    names_to = "Variable",
    values_to = "Value"
  )
p_numvars <- ggplot(
  data_numeric_long %>% filter(!is.na(Value)),
  aes(x = Value)
) +
  geom_histogram(bins = 30, fill = "steelblue", color = "white") +
  facet_wrap(~ Variable, scales = "free") +

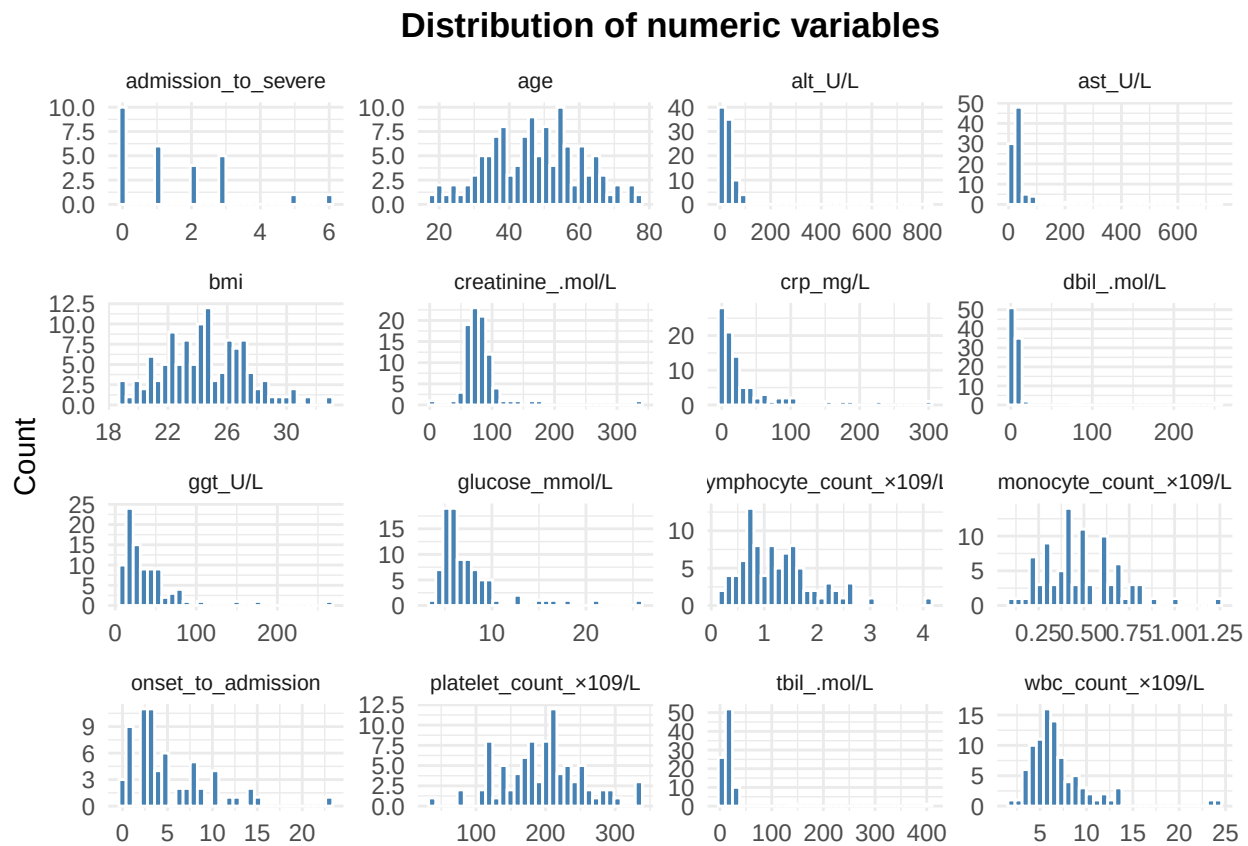
```

```

labs(
  title = "Distribution of numeric variables",
  x = NULL,
  y = "Count"
) +
theme_minimal() +
theme(
  strip.text = element_text(size = 8),
  plot.title = element_text(hjust = 0.5, face = "bold")
)

p_numvars

```



```

ggsave(
  filename = "data/processed_data/numeric_variables_distribution.png",
  plot = p_numvars,
  width = 6,
  height = 4,
  dpi = 300
)

```

```

# - Categorical variables: frequency tables
# Regarding categorical variables, we have already done the distribution analysis in the previous section
# But we can also show here the frequency tables and the bar plots can be checked at data balancing section
categorical_summary <- lapply(data_categorical, function(x) {

```



```

freq <- table(x)
perc <- prop.table(freq) * 100
data.frame(
  Category = names(freq),
  Frequency = as.numeric(freq),
  Percentage = round(as.numeric(perc), 2)
)
})

categorical_summary

```

```

## $sex
##   Category Frequency Percentage
## 1   Female         41       34.75
## 2    Male         77       65.25
##
## $group
##   Category Frequency Percentage
## 1 Healthy Control      28       23.73
## 2   Non-COVID-19      25       21.19
## 3   Non-severe       37       31.36
## 4      Severe        28       23.73

```

```

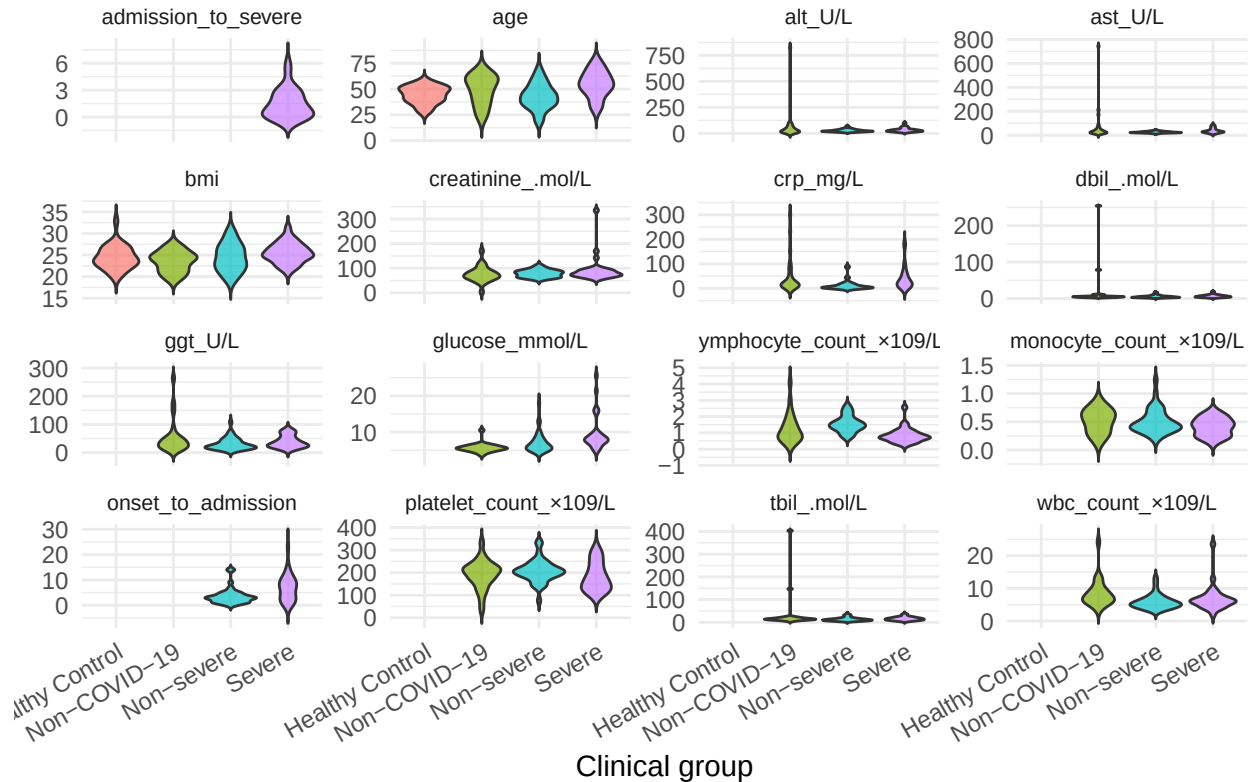
# Bivariate Analysis
# - Comparisons between key numeric variables and clinical group
data_numeric_long <- data_cleaned %>%
  select(all_of(c(numeric_cols, "group"))) %>%
  pivot_longer(
    cols = all_of(numeric_cols),
    names_to = "Variable",
    values_to = "Value"
  )

p_numvars_cli <- ggplot(
  data_numeric_long %>% filter(!is.na(Value)),
  aes(x = group, y = Value, fill = group)
) +
  geom_violin(trim = FALSE, alpha = 0.7) +
  facet_wrap(~ Variable, scales = "free_y") +
  labs(
    title = "Distribution of numeric variables by clinical group",
    x = "Clinical group",
    y = NULL
  ) +
  theme_minimal() +
  theme(
    legend.position = "none",
    strip.text = element_text(size = 8),
    plot.title = element_text(hjust = 0.5, face = "bold"),
    axis.text.x = element_text(angle = 30, hjust = 1)
  )

p_numvars_cli

```

## Distribution of numeric variables by clinical group



```
ggsave(
  filename = "data/processed_data/numeric_variables_pergroup_distribution.png",
  plot = p_numvars_cli,
  width = 6,
  height = 4,
  dpi = 300
)

# Detect Outliers
# This will be the next step but will be done in the Exercise 2.2 section
```

- A brief written summary of notable patterns, anomalies, or data quality issues

The exploratory data analysis revealed several notable patterns. First, the dataset shows a moderate imbalance in terms of sex, with a higher proportion of male patients (65%).

Several biomarkers, including CRP, ALT, AST, and bilirubin levels, show highly skewed distributions with the presence of extreme values.

The distributions of variables by clinical group indicate that patients in the severe category tend to present higher levels of these biomarkers, whereas lymphocyte counts are generally lower, what can be associated with disease severity.

Regarding data quality, the analysis of missing values indicates that they are not randomly distributed but instead reflect the study design. For example, many biomarkers were measured exclusively in COVID-19 patients. Therefore, missing values are generally not problematic, with the exception of variables such as BMI and CRP, where missingness appears unrelated to the study design and required imputation.

## Exercise 2

### Exercise 2.1

Reproduce Supplementary Figure 1 using the data without modifying or removing any observations. Use the same variables and groupings as in the original figure.

```
# =====  
# Exercise 2.1 - Reproduce Supplementary Figure 1 (BAR PLOTS) -----> version with breaks in the y-axis  
# =====  
  
library(ggpubr)  
library(ggplot2)  
library(dplyr)  
library(ggbreak)  
library(ggplotify)  
  
# 1. Prepare the data  
data_fig1 <- data_cleaned %>%  
  filter(group %in% c("Non-COVID-19", "Non-severe", "Severe")) %>%  
  mutate(group = factor(group, levels = c("Non-COVID-19", "Non-severe", "Severe")))  
  
# 2. Define colors and shapes  
my_colors <- c("Non-COVID-19" = "#567FD0", "Non-severe" = "#FFB04F", "Severe" = "#CB4838")  
# Shapes: 16=Circle, 15=Square, 17=Triangle  
my_shapes <- c("Non-COVID-19" = 16, "Non-severe" = 15, "Severe" = 17)  
  
# 3. Define comparisons  
my_comparisons <- list(  
  c("Non-COVID-19", "Non-severe"),  
  c("Non-severe", "Severe"),  
  c("Non-COVID-19", "Severe")  
)  
  
# 4. Functions to calculate mean and error bars  
# mean ± SD  
mean_sd_trunc <- function(x) {  
  m <- mean(x, na.rm = TRUE)  
  s <- sd(x, na.rm = TRUE)  
  
  data.frame(  
    y = m,  
    ymin = pmax(0, m - s), # truncado  
    ymax = m + s  
  )  
}  
  
# mean ± SEM  
mean_sem <- function(x) {  
  m <- mean(x, na.rm = TRUE)  
  s <- sd(x, na.rm = TRUE) / sqrt(sum(!is.na(x)))  
  data.frame(  
    y = m,  
    ymin = pmax(0, m - s),  
    ymax = m + s  
  )  
}
```

```

)
}

# 5. Function to create the bar plot
create_barplot <- function(data, y_var, title, y_label, break_point = NULL, break_scale = NULL, compress

p <- ggplot(data, aes(x = group, y = .data[[y_var]], color = group)) +

  # 5.1. The bars: white fill, colored border
  geom_bar(stat = "summary", fun = "mean",
           fill = "white", # The fill is white
           width = 0.5,    # The width of the bar
           linewidth = 0.8) + # The width of the border line

  # 5.2. Error bars (Mean +/- Standard Error of the Mean)
  stat_summary(fun.data = mean_se, geom = "errorbar",
              width = 0.2, linewidth = 0.8) +

  # 5.3. Individual points (Jitter) with different SHAPES
  # Map 'shape' to group as well
  geom_jitter(aes(shape = group),
             width = 0.25, size = 1.5, alpha = 0.8) +

  # 5.4. Significance bars with t-test (Welch's test)
  stat_compare_means(comparisons = my_comparisons,
                    method = "t.test",
                    method.args = list(var.equal = FALSE),
                    label = "p.signif",
                    symnum.args = list(cutpoints = c(0, 0.001, 0.01, 0.05, 1),
                                       symbols = c("***", "**", "*", "ns")),
                    hide.ns = TRUE, # Hide non-significant for clarity
                    color = "black", # Significance bars -> black
                    vjust = 0.8) +

  # 5.5. Aesthetics and scales
  scale_color_manual(values = my_colors) +
  scale_shape_manual(values = my_shapes) +
  labs(title = title, y = y_label, x = NULL) +
  theme_classic() +
  theme(
    legend.position = "none",
    plot.title = element_text(hjust = 0.5, size = 11, face = "bold"),
    axis.text.x = element_blank(), # Remove x-axis text
    axis.ticks.x = element_blank(),
    axis.line = element_line(linewidth = 0.8) # Slightly thicker axes
  )

# Adjust y-axis limits
p <- p + scale_y_continuous(expand = expansion(mult = c(0, 0.1)))

# Add y-axis break if specified
if (!is.null(break_point)) {
  if (!is.null(break_scale)) {

```

```

    p <- p +
      scale_y_break(
        breaks = c(break_point, break_scale),
        scales = compress_scale
      )
  }else{
    p <- p +
      scale_y_cut(
        breaks = break_point,
        scales = compress_scale
      )
  }
}

return(p)
}

# 6. Create the 12 bar plots with the function

p1 <- create_barplot(data_fig1, "wbc_count_×109/L", "White blood cell (WBC)", "Count (×109/L)")
p2 <- create_barplot(data_fig1, "lymphocyte_count_×109/L", "Lymphocyte", "Count (×109/L)")
p3 <- create_barplot(data_fig1, "monocyte_count_×109/L", "Monocyte", "Count (×109/L)")
p4 <- create_barplot(data_fig1, "platelet_count_×109/L", "Platelet", "Count (×109/L)")
p5 <- create_barplot(data_fig1, "crp_mg/L", "C-reactive protein (CRP)", "mg/L", break_point = 50, compress_scale = 10)
p6 <- create_barplot(data_fig1, "alt_U/L", "Alanine aminotransferase (ALT)", "U/L", break_point = 60, compress_scale = 10)
p7 <- create_barplot(data_fig1, "ast_U/L", "Aspartate aminotransferase (AST)", "U/L", break_point = 100, compress_scale = 10)
p8 <- create_barplot(data_fig1, "ggt_U/L", "Glutamyltransferase (GGT)", "U/L", break_point = 100, compress_scale = 10)
p9 <- create_barplot(data_fig1, "tbil_mol/L", "Total bilirubin (TBIL)", "mol/L", break_point = 50, break_scale = 10)
p10 <- create_barplot(data_fig1, "dbil_mol/L", "Direct bilirubin (DBIL)", "mol/L", break_point = 20, break_scale = 10)
p11 <- create_barplot(data_fig1, "creatinine_mol/L", "Creatinine", "mol/L", break_point = 180, break_scale = 10)
p12 <- create_barplot(data_fig1, "glucose_mmol/L", "Glucose", "mmol/L", break_point = 15, compress_scale = 10)

# Libraries
library(aplot)
library(cowplot)
library(patchwork)

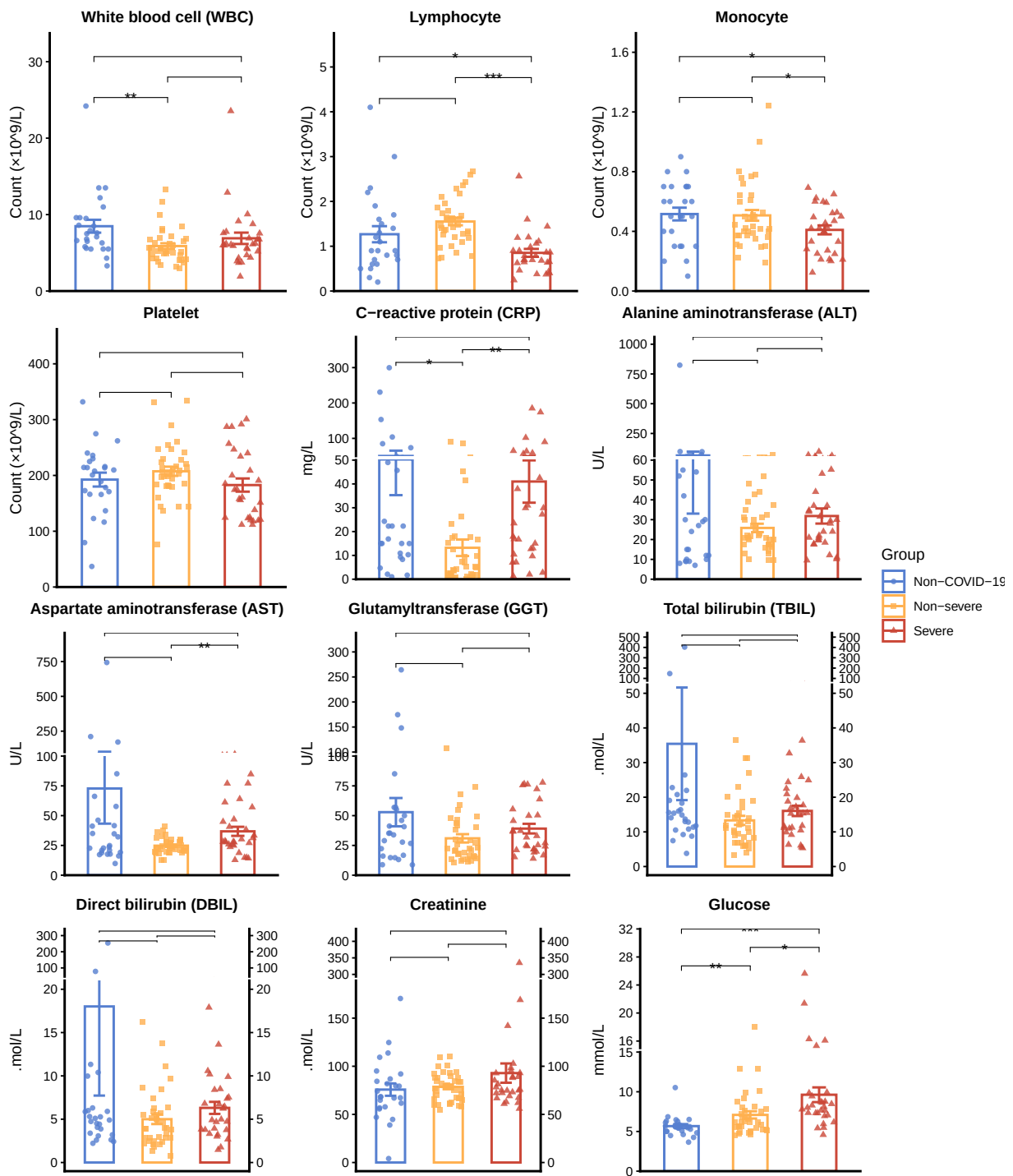
# 7. Create a list with our 12 plots (the ones with the break)
lista_plots <- list(
  p1, p2, p3,
  p4, p5, p6,
  p7, p8, p9,
  p10, p11, p12
)

# 8. Extract the shared legend from one of the plots
shared_legend <- get_legend(
  p1 + theme(legend.position = "right") + labs(color = "Group", shape = "Group")
)

# 9. Use plot_list to create the grid
# Align the inner panels (including breaks)
# so that all axes and drawing areas measure the same.

```

```
figure_grid <- plot_list(  
  gglist = lista_plots,  
  ncol = 3,  
  nrow = 4  
)  
  
figure_s1_final <- wrap_elements(figure_grid) + shared_legend +  
  plot_layout(widths = c(1, 0.1)) # 1 part for the grid, 0.1 for the legend  
  
print(figure_s1_final)
```



```
png("data/processed_data/Supplementary_Figure_1_breaks.png",
    width = 1000, height = 1200, res = 150)
print(figure_s1_final)
dev.off()
```

```
## pdf
```

## 2

```
# =====  
# Exercise 2.1 - Reproduce Supplementary Figure 1 (BAR PLOTS) -----> version without breaks in the y-axis  
# =====  
  
library(ggpubr)  
library(ggplot2)  
library(dplyr)  
  
# 1. Prepare the data  
data_fig1 <- data_cleaned %>%  
  filter(group %in% c("Non-COVID-19", "Non-severe", "Severe")) %>%  
  mutate(group = factor(group, levels = c("Non-COVID-19", "Non-severe", "Severe")))  
  
# 2. Define colors and shapes  
my_colors <- c("Non-COVID-19" = "#567FD0", "Non-severe" = "#FFB04F", "Severe" = "#CB4838")  
# Shapes: 16=Circle, 15=Square, 17=Triangle  
my_shapes <- c("Non-COVID-19" = 16, "Non-severe" = 15, "Severe" = 17)  
  
# 3. Define comparisons  
my_comparisons <- list(  
  c("Non-COVID-19", "Non-severe"),  
  c("Non-severe", "Severe"),  
  c("Non-COVID-19", "Severe")  
)  
  
# 4. Function to create the bar plot  
create_barplot <- function(data, y_var, title, y_label) {  
  
  ggplot(data, aes(x = group, y = .data[[y_var]], color = group)) +  
  
    # 4.1. The bars: white fill, colored border  
    geom_bar(stat = "summary", fun = "mean",  
            fill = "white", # White fill  
            width = 0.5, # Bar width  
            linewidth = 0.8) + # Border line thickness  
  
    # 4.2. Error bars (Mean +/- Standard Error of the Mean)  
    stat_summary(fun.data = "mean_sd", geom = "errorbar",  
                width = 0.2, size = 0.8) +  
  
    # 4.3. Individual points (Jitter) with different SHAPES  
    # Map 'shape' to group as well  
    geom_jitter(aes(shape = group),  
                width = 0.25, size = 1.5, alpha = 0.8) +  
  
    # 4.4. Significance bars with t-test (Welch's test)  
    stat_compare_means(comparisons = my_comparisons,  
                       method = "t.test",  
                       method.args = list(var.equal = FALSE),  
                       label = "p.signif",  
                       symnum.args = list(cutpoints = c(0, 0.001, 0.01, 0.05, 1),  
                                           symbols = c("***", "**", "*", "ns"))),  
  }
```



```

        hide.ns = TRUE, # Hide non-significant for clarity
        color = "black", # Significance bars are usually black
        vjust = 0.5) +

# 4.5. Aesthetics and scales
scale_color_manual(values = my_colors) +
scale_shape_manual(values = my_shapes) +

labs(title = title, y = y_label, x = NULL) +
theme_classic() +
theme(
  legend.position = "none",
  plot.title = element_text(hjust = 0.5, size = 11, face = "bold"),
  axis.text.x = element_blank(), # Remove x-axis text
  axis.ticks.x = element_blank(),
  axis.line = element_line(linewidth = 0.8) # Slightly thicker axes
)
}

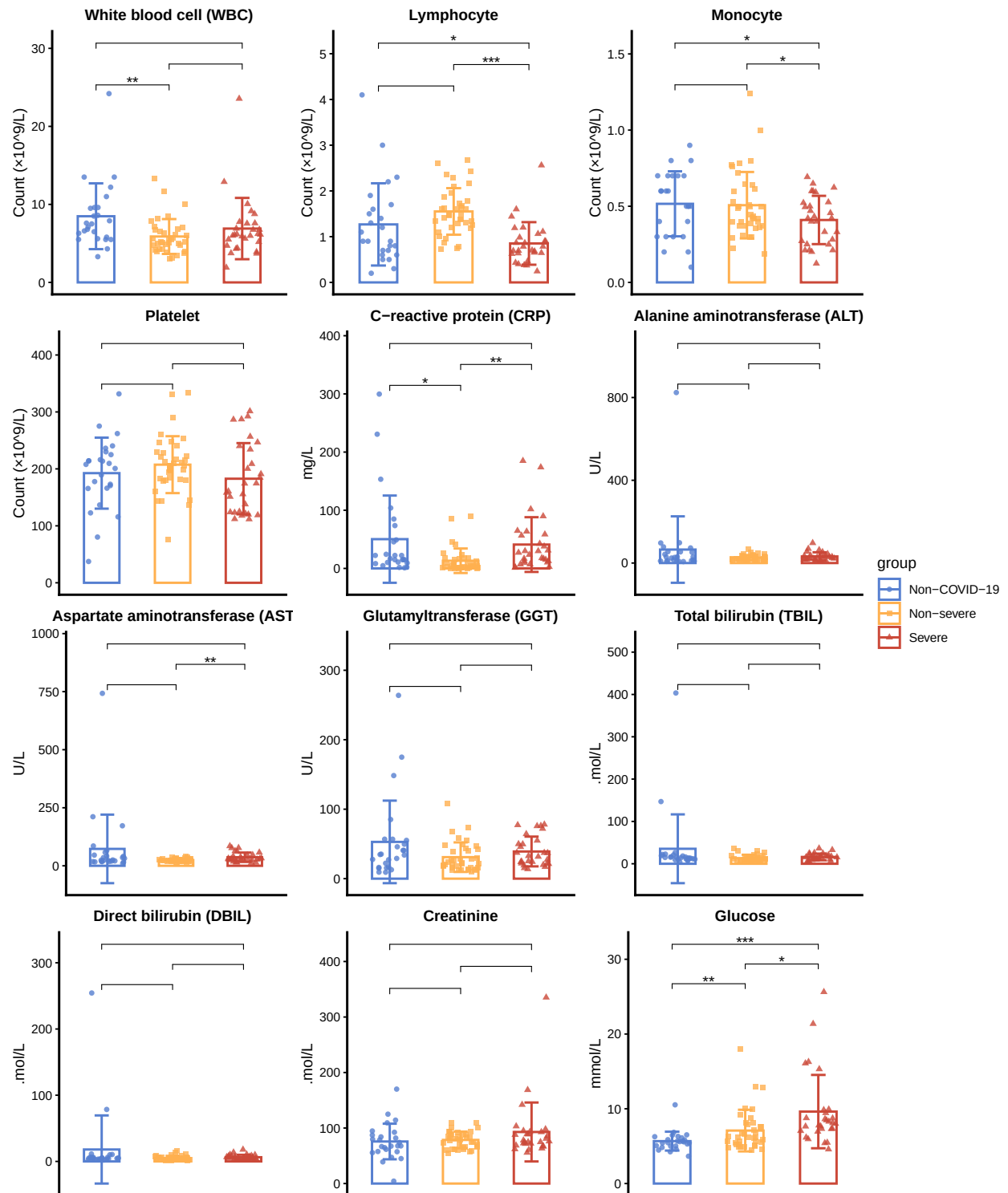
# 5. Generate the 12 plots with the function

p1 <- create_barplot(data_fig1, "wbc_count_x109/L", "White blood cell (WBC)", "Count (×109/L)")
p2 <- create_barplot(data_fig1, "lymphocyte_count_x109/L", "Lymphocyte", "Count (×109/L)")
p3 <- create_barplot(data_fig1, "monocyte_count_x109/L", "Monocyte", "Count (×109/L)")
p4 <- create_barplot(data_fig1, "platelet_count_x109/L", "Platelet", "Count (×109/L)")
p5 <- create_barplot(data_fig1, "crp_mg/L", "C-reactive protein (CRP)", "mg/L")
p6 <- create_barplot(data_fig1, "alt_U/L", "Alanine aminotransferase (ALT)", "U/L")
p7 <- create_barplot(data_fig1, "ast_U/L", "Aspartate aminotransferase (AST)", "U/L")
p8 <- create_barplot(data_fig1, "ggt_U/L", "Glutamyltransferase (GGT)", "U/L")
p9 <- create_barplot(data_fig1, "tbil_mol/L", "Total bilirubin (TBIL)", "mol/L")
p10 <- create_barplot(data_fig1, "dbil_mol/L", "Direct bilirubin (DBIL)", "mol/L")
p11 <- create_barplot(data_fig1, "creatinine_mol/L", "Creatinine", "mol/L")
p12 <- create_barplot(data_fig1, "glucose_mmol/L", "Glucose", "mmol/L")

# 6. Combine and display
figure_s1_bars <- ggarrange(p1, p2, p3, p4, p5, p6, p7, p8, p9, p10, p11, p12,
  ncol = 3, nrow = 4,
  common.legend = TRUE, legend = "right") # Legend on the right as reference

print(figure_s1_bars)

```



```
png("data/processed_data/Supplementary_Figure_1_no_breaks.png",
     width = 1000, height = 1200, res = 150)
print(figure_s1_bars)
dev.off()
```

```
## pdf
```

## 2

## Exercise 2.2

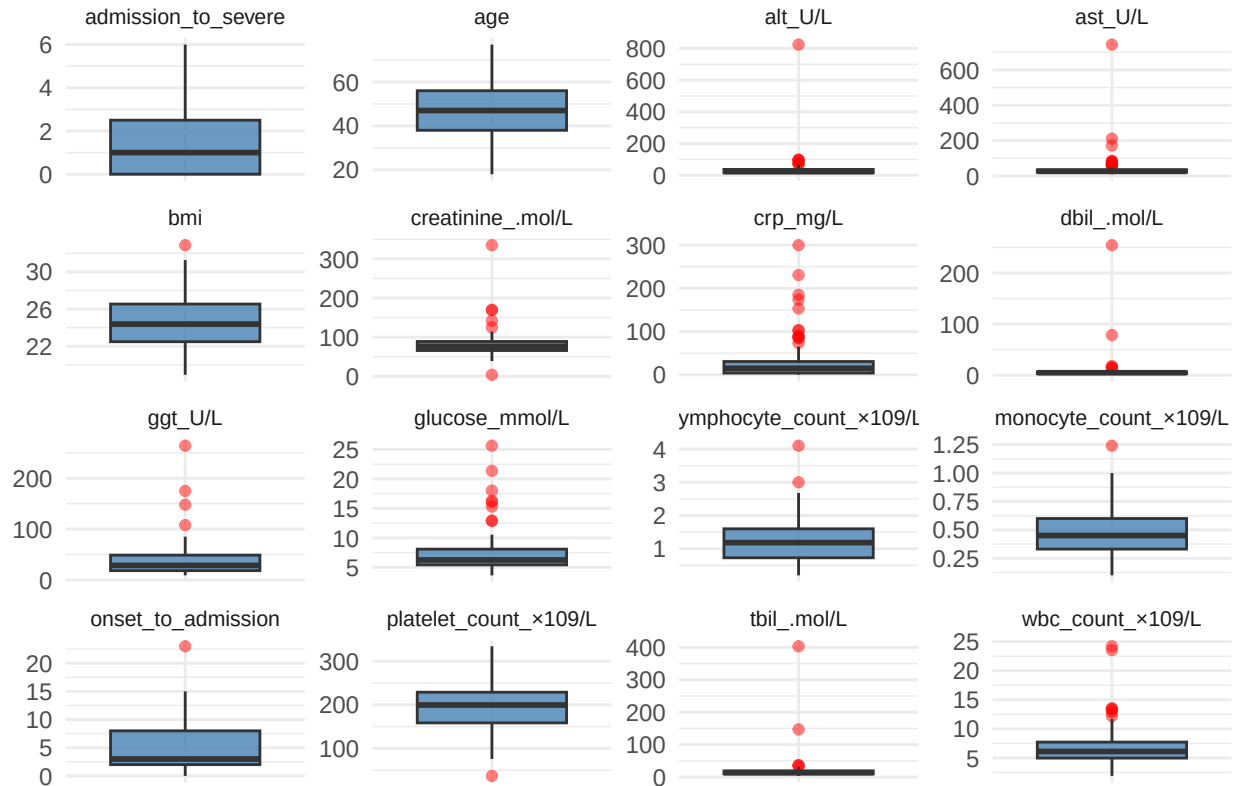
Identify and handle outliers using an appropriate and well-justified method (e.g. removal, transformation, or robust statistics).

First, we detected outliers using boxplots.

```
# Outlier detection using boxplots
p_outliers <- ggplot(
  data_numeric_long %>% filter(!is.na(Value)), # Remove NAs for better visualization
  aes(x = "", y = Value)
) +
  geom_boxplot(
    fill = "steelblue",
    alpha = 0.8,      # Transparency of the boxplot fill
    width = 0.75,     # Width of the boxplot
    outlier.colour = "red", # Color of outliers
    outlier.alpha = 0.5   # Transparency of outliers
  ) +
  facet_wrap(~ Variable, scales = "free_y") +
  labs(
    title = "Outlier detection using boxplots",
    x = NULL,
    y = NULL
  ) +
  theme_minimal() +
  theme(
    strip.text = element_text(size = 8),
    plot.title = element_text(hjust = 0.5, face = "bold"),
    axis.text.x = element_blank(),
    axis.ticks.x = element_blank()
  )

p_outliers
```

## Outlier detection using boxplots



```
ggsave(
  filename = "data/processed_data/outliers_detection.png",
  plot = p_outliers,
  width = 6,
  height = 4,
  dpi = 300
)
```

Then, we obtained a summary of the outliers. The outlier detection function is based in IQR-based criteria because Z-score-based methods identified extreme values primarily in approximately normally distributed variables but with this properties we only have age and BMI. In contrast, IQR-based methods are more effective at detecting extreme values in skewed biomarker distributions so, we considered that is the most appropriate for summarizing biomarker outliers.

```
# Outliers summary
library(outliers)
library(purrr)

# ---- Function to detect outliers (IQR based) ----
detect_outliers_iqr_long <- function(data, var_name, id_col = "patient_id") {

  x <- data[[var_name]]
  ids <- data[[id_col]]

  x_clean <- x[!is.na(x)]
```

```

bp <- boxplot.stats(x_clean)
out_vals <- bp$out

if (length(out_vals) == 0) {
  return(tibble())
}

tibble(
  patient_id = ids[x %in% out_vals],
  Variable = var_name,
  Outlier_value = x[x %in% out_vals]
) %>%
  mutate(
    Mean_variable = round(mean(x_clean), 2),
    SD_variable = round(sd(x_clean), 2)
  ) %>%
  select(
    patient_id,
    Variable,
    Outlier_value,
    Mean_variable,
    SD_variable
  )
}

# ---- Apply to all numeric variables ----
outliers_iqr_long_table <- map_dfr(
  numeric_cols,
  ~ detect_outliers_iqr_long(data_cleaned, .x)
)

# ---- Display nicely formatted table ----
outliers_iqr_long_table %>%
  mutate(
    Outlier_value = round(Outlier_value, 2)
  ) %>%
  kable(
    caption = "Detailed list of outliers detected using the IQR criterion",
    align = "llrrr"
  ) %>%
  kable_styling(
    bootstrap_options = c("striped", "hover", "condensed"),
    full_width = FALSE,
    position = "center"
  )

```

Table 2: Detailed list of outliers detected using the IQR criterion

| patient_id | Variable         | Outlier_value | Mean_variable | SD_variable |
|------------|------------------|---------------|---------------|-------------|
| HC27       | bmi              | 32.87         | 24.50         | 2.79        |
| XG38       | wbc_count_×109/L | 12.89         | 6.92          | 3.55        |
| XG46       | wbc_count_×109/L | 23.55         | 6.92          | 3.55        |

|        |                                  |        |        |       |
|--------|----------------------------------|--------|--------|-------|
| X2-28  | wbc_count_ $\times 109/L$        | 13.29  | 6.92   | 3.55  |
| JBDZ1  | wbc_count_ $\times 109/L$        | 12.20  | 6.92   | 3.55  |
| JBDZ3  | wbc_count_ $\times 109/L$        | 13.50  | 6.92   | 3.55  |
| JBDZ18 | wbc_count_ $\times 109/L$        | 24.20  | 6.92   | 3.55  |
| JBDZ19 | wbc_count_ $\times 109/L$        | 13.50  | 6.92   | 3.55  |
| JBDZ4  | lymphocyte_count_ $\times 109/L$ | 3.00   | 1.26   | 0.69  |
| JBDZ22 | lymphocyte_count_ $\times 109/L$ | 4.10   | 1.26   | 0.69  |
| XG1    | monocyte_count_ $\times 109/L$   | 1.24   | 0.48   | 0.20  |
| JBDZ6  | platelet_count_ $\times 109/L$   | 37.00  | 195.61 | 57.94 |
| XG3    | crp_mg/L                         | 89.90  | 32.15  | 51.31 |
| XG39   | crp_mg/L                         | 101.89 | 32.15  | 51.31 |
| XG41   | crp_mg/L                         | 185.00 | 32.15  | 51.31 |
| XG43   | crp_mg/L                         | 89.80  | 32.15  | 51.31 |
| XG44   | crp_mg/L                         | 173.90 | 32.15  | 51.31 |
| X2-22  | crp_mg/L                         | 85.40  | 32.15  | 51.31 |
| JBDZ7  | crp_mg/L                         | 103.90 | 32.15  | 51.31 |
| JBDZ11 | crp_mg/L                         | 299.70 | 32.15  | 51.31 |
| JBDZ15 | crp_mg/L                         | 153.30 | 32.15  | 51.31 |
| JBDZ17 | crp_mg/L                         | 230.70 | 32.15  | 51.31 |
| JBDZ21 | crp_mg/L                         | 84.90  | 32.15  | 51.31 |
| JBDZ22 | crp_mg/L                         | 73.60  | 32.15  | 51.31 |
| XG26   | alt_U/L                          | 69.00  | 38.62  | 86.17 |
| XG31   | alt_U/L                          | 97.00  | 38.62  | 86.17 |
| JBDZ2  | alt_U/L                          | 824.00 | 38.62  | 86.17 |
| JBDZ4  | alt_U/L                          | 73.00  | 38.62  | 86.17 |
| JBDZ7  | alt_U/L                          | 97.00  | 38.62  | 86.17 |
| JBDZ11 | alt_U/L                          | 98.00  | 38.62  | 86.17 |
| JBDZ16 | alt_U/L                          | 78.00  | 38.62  | 86.17 |
| XG26   | ast_U/L                          | 77.00  | 41.86  | 80.07 |
| XG39   | ast_U/L                          | 77.00  | 41.86  | 80.07 |
| XG41   | ast_U/L                          | 64.00  | 41.86  | 80.07 |
| XG43   | ast_U/L                          | 61.00  | 41.86  | 80.07 |
| XG44   | ast_U/L                          | 85.00  | 41.86  | 80.07 |
| XG45   | ast_U/L                          | 57.00  | 41.86  | 80.07 |
| JBDZ1  | ast_U/L                          | 172.00 | 41.86  | 80.07 |
| JBDZ2  | ast_U/L                          | 743.00 | 41.86  | 80.07 |
| JBDZ4  | ast_U/L                          | 58.00  | 41.86  | 80.07 |
| JBDZ6  | ast_U/L                          | 66.00  | 41.86  | 80.07 |
| JBDZ7  | ast_U/L                          | 85.00  | 41.86  | 80.07 |
| JBDZ11 | ast_U/L                          | 211.00 | 41.86  | 80.07 |
| X2-28  | ggt_U/L                          | 108.00 | 39.59  | 36.82 |
| JBDZ2  | ggt_U/L                          | 148.00 | 39.59  | 36.82 |
| JBDZ4  | ggt_U/L                          | 264.00 | 39.59  | 36.82 |
| JBDZ10 | ggt_U/L                          | 175.00 | 39.59  | 36.82 |
| XG31   | tbil_mol/L                       | 32.70  | 20.31  | 43.76 |
| XG35   | tbil_mol/L                       | 36.40  | 20.31  | 43.76 |
| XG25   | tbil_mol/L                       | 36.50  | 20.31  | 43.76 |
| JBDZ2  | tbil_mol/L                       | 403.40 | 20.31  | 43.76 |
| JBDZ6  | tbil_mol/L                       | 146.90 | 20.31  | 43.76 |
| XG16   | dbil_mol/L                       | 13.80  | 9.02   | 27.46 |

|        |                    |        |       |       |
|--------|--------------------|--------|-------|-------|
| XG31   | dbil_ mol/L        | 17.90  | 9.02  | 27.46 |
| XG35   | dbil_ mol/L        | 13.60  | 9.02  | 27.46 |
| XG25   | dbil_ mol/L        | 16.20  | 9.02  | 27.46 |
| JBDZ2  | dbil_ mol/L        | 254.40 | 9.02  | 27.46 |
| JBDZ6  | dbil_ mol/L        | 78.50  | 9.02  | 27.46 |
| XG29   | creatinine_ mol/L  | 335.00 | 82.26 | 35.71 |
| XG43   | creatinine_ mol/L  | 142.00 | 82.26 | 35.71 |
| XG44   | creatinine_ mol/L  | 169.00 | 82.26 | 35.71 |
| JBDZ3  | creatinine_ mol/L  | 4.00   | 82.26 | 35.71 |
| JBDZ11 | creatinine_ mol/L  | 170.00 | 82.26 | 35.71 |
| JBDZ21 | creatinine_ mol/L  | 125.00 | 82.26 | 35.71 |
| XG28   | glucose_ mmol/L    | 16.28  | 7.48  | 3.65  |
| XG36   | glucose_ mmol/L    | 25.63  | 7.48  | 3.65  |
| XG37   | glucose_ mmol/L    | 15.31  | 7.48  | 3.65  |
| XG22   | glucose_ mmol/L    | 12.86  | 7.48  | 3.65  |
| XG43   | glucose_ mmol/L    | 21.37  | 7.48  | 3.65  |
| X2-22  | glucose_ mmol/L    | 18.02  | 7.48  | 3.65  |
| X2-28  | glucose_ mmol/L    | 12.94  | 7.48  | 3.65  |
| X2-14  | glucose_ mmol/L    | 16.08  | 7.48  | 3.65  |
| XG45   | onset_to_admission | 23.00  | 5.00  | 4.42  |

Many outliers are associated with biomarkers such as CRP, ALT, AST, bilirubin, and glucose, which is consistent with their highly skewed distributions observed during the analysis.

```
# Remove outliers based on IQR (per variable)

remove_outliers_iqr <- function(data, var) {
  Q1 <- quantile(data[[var]], 0.25, na.rm = TRUE)
  Q3 <- quantile(data[[var]], 0.75, na.rm = TRUE)
  IQR_val <- Q3 - Q1

  lower <- Q1 - 1.5 * IQR_val
  upper <- Q3 + 1.5 * IQR_val

  data %>%
    filter(.data[[var]] >= lower | is.na(.data[[var]])) %>%
    filter(.data[[var]] <= upper | is.na(.data[[var]]))
}

# Apply sequentially to all numeric variables
data_no_outliers <- data_cleaned

for (var in numeric_cols) {
  data_no_outliers <- remove_outliers_iqr(data_no_outliers, var)
}

nrow(data_cleaned)
```

```
## [1] 118
```

```
nrow(data_no_outliers)
```

```
## [1] 74
```

After detecting outliers, we removed them using an IQR-based filtering method to remove observations that fall outside the acceptable range. This results in a substantial reduction of the dataset size (from 118 to 74 observations), indicating that many data is classified as extreme.

A dataset without outliers was created to reproduce the supplementary figure. However, this approach may be aggressive because extreme values in biomedical data can carry meaningful clinical information rather than noise. So, subsequent analyses will rely on the original dataset, where specific potentially noisy observations (individual patients) are evaluated and removed only if justified.

Also, patient JBDZ2 was identified as an extreme observation, showing elevated values in several biomarkers its unusually high measurements suggest that this patient may influence results after normalization.

Reproduce Supplementary Figure 1 after outlier handling and discuss how and why the results change.

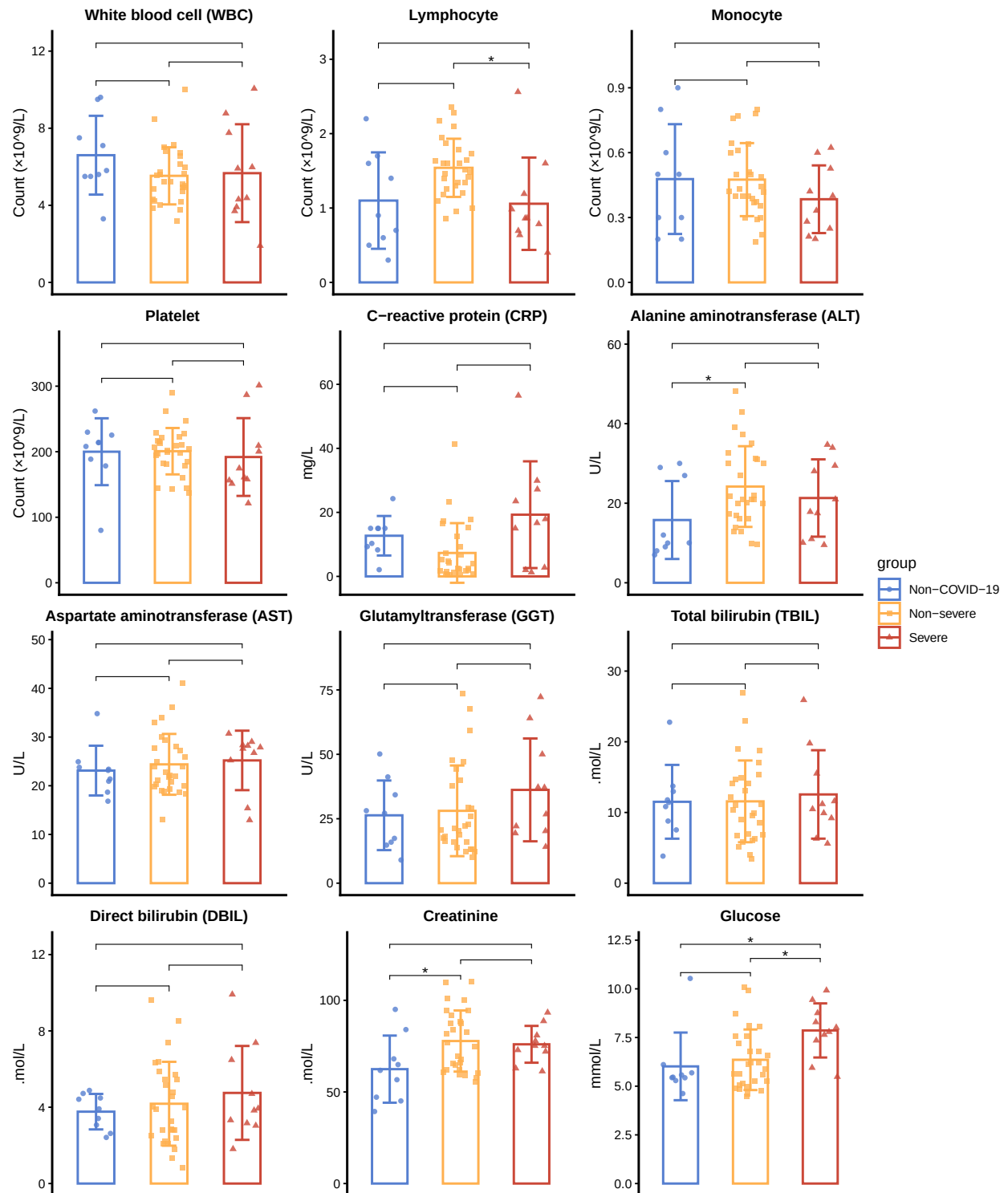
```
data_fig1_no_outliers <- data_no_outliers %>%
  filter(group %in% c("Non-COVID-19", "Non-severe", "Severe")) %>%
  mutate(group = factor(group, levels = c("Non-COVID-19", "Non-severe", "Severe")))

p1 <- create_barplot(data_fig1_no_outliers, "wbc_count_×109/L", "White blood cell (WBC)", "Count (×109/L)")
p2 <- create_barplot(data_fig1_no_outliers, "lymphocyte_count_×109/L", "Lymphocyte", "Count (×109/L)")
p3 <- create_barplot(data_fig1_no_outliers, "monocyte_count_×109/L", "Monocyte", "Count (×109/L)")
p4 <- create_barplot(data_fig1_no_outliers, "platelet_count_×109/L", "Platelet", "Count (×109/L)")
p5 <- create_barplot(data_fig1_no_outliers, "crp_mg/L", "C-reactive protein (CRP)", "mg/L")
p6 <- create_barplot(data_fig1_no_outliers, "alt_U/L", "Alanine aminotransferase (ALT)", "U/L")
p7 <- create_barplot(data_fig1_no_outliers, "ast_U/L", "Aspartate aminotransferase (AST)", "U/L")
p8 <- create_barplot(data_fig1_no_outliers, "ggt_U/L", "Glutamyltransferase (GGT)", "U/L")
p9 <- create_barplot(data_fig1_no_outliers, "tbil_mol/L", "Total bilirubin (TBIL)", "mol/L")
p10 <- create_barplot(data_fig1_no_outliers, "dbil_mol/L", "Direct bilirubin (DBIL)", "mol/L")
p11 <- create_barplot(data_fig1_no_outliers, "creatinine_mol/L", "Creatinine", "mol/L")
p12 <- create_barplot(data_fig1_no_outliers, "glucose_mmol/L", "Glucose", "mmol/L")

figure_s1_no_outliers <- ggarrange(
  p1, p2, p3, p4, p5, p6, p7, p8, p9, p10, p11, p12,
  ncol = 3, nrow = 4,
  common.legend = TRUE, legend = "right"
)

print(figure_s1_no_outliers)
```





```
png("data/processed_data/Supplementary_Figure_1_no_breaks_no_outliers.png",
     width = 1000, height = 1200, res = 150)
print(figure_s1_no_outliers)
dev.off()
```

```
## pdf
```

Comparing the Supplementary Figure 1 before and after the outliers removing, a clear reduction in the scale and dispersion of several biomarkers is observed after outlier removal. In the original figure, some variables such as CRP, ALT, AST, GGT, TBIL, and DBIL exhibited extremely large ranges because of few patients with very high values. After outlier handling, these ranges become narrower which allows a better visualization making group differences clearer.

Importantly, the overall trends across groups remain consistent. Severe patients still tend to show higher levels of some biomarkers (such as CRP and GGT), while Non-COVID-19 patients generally present lower values. This indicates that the main biological differences between groups are not only driven by extreme values.

## Exercise 3

Create a heatmap of the biomarker data. Include group and gender as annotation variables.

Multiple heatmaps are generated to explore how different preprocessing choices affect the visualization and interpretation of the data. Specifically, the heatmaps differ in three main aspects:

- Whether extreme observations (patient JBDZ2) are included or excluded
- Whether a log transformation is applied prior to normalization
- Whether scaling is performed manually (z-score) or within the heatmap function

These variations allow for a sensitivity analysis, helping to assess the robustness of the observed clustering patterns and ensuring that results are not driven by a small number of extreme values or by specific preprocessing decisions.

```
# ---- 1. Install and load necessary library ----
if (!require("pheatmap")) install.packages("pheatmap")
library(pheatmap)

# ---- 2. Define biomarkers for analysis ----
# We select clinical laboratory variables, excluding time-related metrics
biomarkers_names <- c("wbc_count_×109/L", "lymphocyte_count_×109/L",
                      "monocyte_count_×109/L", "platelet_count_×109/L",
                      "crp_mg/L", "alt_U/L", "ast_U/L", "ggt_U/L", "tbil_mol/L",
                      "dbil_mol/L", "creatinine_mol/L", "glucose_mmol/L")

# ---- 3. Prepare the data matrix ----
# We filter for patients who have clinical data available
# Note: Healthy controls lack these biomarkers (NAs) so, will be discarded
data_heatmap <- data_cleaned %>%
  drop_na(all_of(biomarkers_names))

# Normalization of the data (Z-score)
# Convert the selected columns into a numeric matrix for pheatmap
mat_raw <- data_heatmap %>%
  select(all_of(biomarkers_names)) %>%
  as.matrix()

mat_normalized <- scale(mat_raw)      # Standardization: (x - mean) / sd
```

```

# Convert to data frame to check the normalized values
df_normalized <- as.data.frame(mat_normalized)
df_normalized$patient_id <- data_heatmap$patient_id

# Assign patient IDs as row names to maintain traceability
rownames(mat_normalized) <- data_heatmap$patient_id

# Force symmetric color scale so that 0 is exactly white
max_abs <- max(abs(mat_normalized), na.rm = TRUE)

# Define breaks centered at 0
breaks <- seq(-max_abs, max_abs, length.out = 101)

# ---- 4. Create Metadata Annotations (Group and Sex) ----
# This dataframe defines the color bars shown at the top/side of the heatmap
anno_df <- data_heatmap %>%
  select(group, sex) %>%
  as.data.frame()

# Row names must match the matrix row names
rownames(anno_df) <- data_heatmap$patient_id

# Define specific color palettes for the clinical groups and sex
anno_colors <- list(
  group = c("Non-COVID-19" = "#66c2a5", "Non-severe" = "#fdae61", "Severe" = "#d53e4f"),
  sex = c("Male" = "steelblue", "Female" = "salmon")
)

```

This first heatmap shows the biomarker profiles across patients using manual z-score normalization without prior transformation.

```

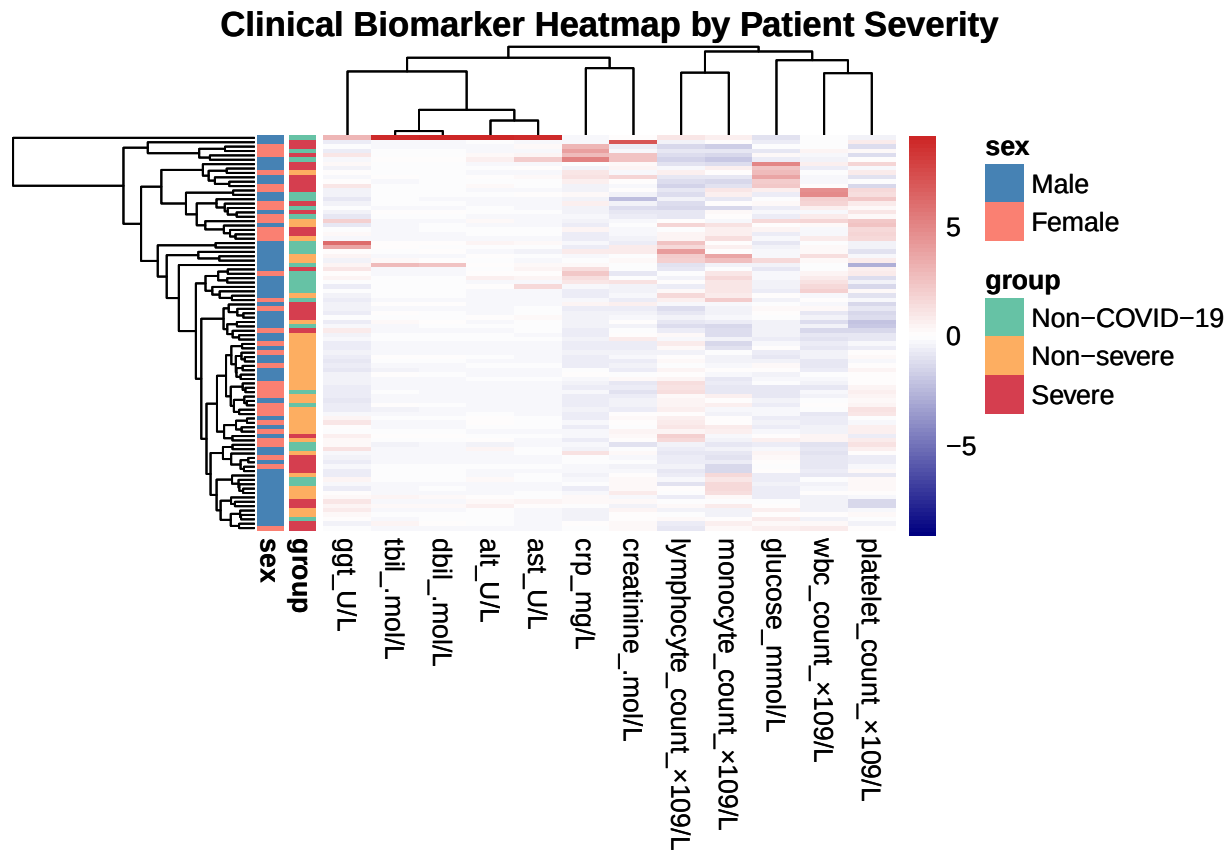
# ---- 5. Generate the Heatmap ----
hm_manualz <- pheatmap(
  mat_normalized,
  cluster_rows = TRUE,          # Cluster patients with similar clinical profiles
  cluster_cols = TRUE,          # Cluster biomarkers that behave similarly
  scale = "none",               # We already normalized manually
  annotation_row = anno_df,     # Add group and sex sidebars
  annotation_colors = anno_colors,
  show_rownames = FALSE,        # Hide individual IDs for better readability

  treeheight_row = 90,          # Increases the width of the left dendrogram
  treeheight_col = 30,          # Height of the top dendrogram
  fontsize_col = 10,
  main = "Clinical Biomarker Heatmap by Patient Severity",

  # Color palette
  color = colorRampPalette(c("navy", "white", "firebrick3"))(100),
  breaks = breaks,
  border_color = NA
)

hm_manualz

```



```
ggsave(
  filename = "data/processed_data/heatmap_manualnorm.png",
  plot = hm_manualz,
  width = 6,
  height = 4,
  dpi = 300
)
```

Function to create the subsequent heatmaps:

```
make_biomarker_heatmap <- function(data, log_transform = TRUE, title_suffix = "") {

  # ---- 2. Define biomarkers for analysis ----
  # We select clinical laboratory variables, excluding time-related metrics
  biomarkers_names <- c("wbc_count_x109/L", "lymphocyte_count_x109/L",
    "monocyte_count_x109/L", "platelet_count_x109/L",
    "crp_mg/L", "alt_U/L", "ast_U/L", "ggt_U/L", "tbil_mol/L",
    "dbil_mol/L", "creatinine_mol/L", "glucose_mmol/L")

  # ---- 3. Prepare the data matrix ----
  # We filter for patients who have clinical data available
  # Note: Healthy controls lack these biomarkers (NAs) so, will be discarded
  data_heatmap <- data %>%
    drop_na(all_of(biomarkers_names))
}
```

```

# Normalization of the data (Z-score)
# Convert the selected columns into a numeric matrix for pheatmap
mat_raw <- data_heatmap %>%
  select(all_of(biomarkers_names)) %>%
  as.matrix()

if (log_transform){
  mat_raw <- log1p(mat_raw)
  norm_label <- "log1p + z-score"
}else{
  norm_label <- "z-score"
}

mat_normalized <- scale(mat_raw)    # Standardization: (x - mean) / sd

# Assign patient IDs as row names to maintain traceability
rownames(mat_normalized) <- data_heatmap$patient_id

# Force symmetric color scale so that 0 is exactly white
max_abs <- max(abs(mat_normalized), na.rm = TRUE)

# Define breaks centered at 0
breaks <- seq(-max_abs, max_abs, length.out = 101)

# ---- 4. Create Metadata Annotations (Group and Sex) ----
# This dataframe defines the color bars shown at the top/side of the heatmap
anno_df <- data_heatmap %>%
  select(group, sex) %>%
  as.data.frame()

# Row names must match the matrix row names
rownames(anno_df) <- data_heatmap$patient_id

# Define specific color palettes for the clinical groups and sex
anno_colors <- list(
  group = c("Non-COVID-19" = "#66c2a5", "Non-severe" = "#fdae61", "Severe" = "#d53e4f"),
  sex = c("Male" = "steelblue", "Female" = "salmon")
)

# ---- 5. Generate the Heatmap ----
pheatmap(
  mat_normalized,
  cluster_rows = TRUE,          # Cluster patients with similar clinical profiles
  cluster_cols = TRUE,          # Cluster biomarkers that behave similarly
  scale = "none",               # We already normalized manually
  annotation_row = anno_df,     # Add group and sex sidebars
  annotation_colors = anno_colors,
  show_rownames = FALSE,        # Hide individual IDs for better readability

  treeheight_row = 90,          # Increases the width of the left dendrogram
  treeheight_col = 30,          # Height of the top dendrogram
  fontsize_col = 10,
  main = paste(

```

```

    "Clinical Biomarker Heatmap by Patient Severity",
    title_suffix,
    paste0("(", norm_label, ")")
  ),

  # Color palette
  color = colorRampPalette(c("navy", "white", "firebrick3"))(100),
  breaks = breaks,
  border_color = NA
)
}

make_biomarker_heatmap_woPrevScale <- function(data, title_suffix = "") {

  # ---- 2. Define biomarkers for analysis ----
  # We select clinical laboratory variables, excluding time-related metrics
  biomarkers_names <- c("wbc_count_x109/L", "lymphocyte_count_x109/L",
                        "monocyte_count_x109/L", "platelet_count_x109/L",
                        "crp_mg/L", "alt_U/L", "ast_U/L", "ggt_U/L", "tbil_mol/L",
                        "dbil_mol/L", "creatinine_mol/L", "glucose_mmol/L")

  # ---- 3. Prepare the data matrix ----
  # We filter for patients who have clinical data available
  # Note: Healthy controls lack these biomarkers (NAs) so, will be discarded
  data_heatmap <- data %>%
    drop_na(all_of(biomarkers_names))

  # Normalization of the data (Z-score)
  # Convert the selected columns into a numeric matrix for pheatmap
  mat_raw <- data_heatmap %>%
    select(all_of(biomarkers_names)) %>%
    as.matrix()

  # Assign patient IDs as row names to maintain traceability
  rownames(mat_raw) <- data_heatmap$patient_id

  # ---- 4. Create Metadata Annotations (Group and Sex) ----
  # This dataframe defines the color bars shown at the top/side of the heatmap
  anno_df <- data_heatmap %>%
    select(group, sex) %>%
    as.data.frame()

  # Row names must match the matrix row names
  rownames(anno_df) <- data_heatmap$patient_id

  # Define specific color palettes for the clinical groups and sex
  anno_colors <- list(
    group = c("Non-COVID-19" = "#66c2a5", "Non-severe" = "#fdae61", "Severe" = "#d53e4f"),
    sex = c("Male" = "steelblue", "Female" = "salmon")
  )

  # ---- 5. Generate the Heatmap ----
  pheatmap(

```

```

mat_raw,
cluster_rows = TRUE,      # Cluster patients with similar clinical profiles
cluster_cols = TRUE,      # Cluster biomarkers that behave similarly
scale = "column",        # We already normalized manually
annotation_row = anno_df, # Add group and sex sidebars
annotation_colors = anno_colors,
show_rownames = FALSE,    # Hide individual IDs for better readability

treeheight_row = 90,      # Increases the width of the left dendrogram
treeheight_col = 30,      # Height of the top dendrogram
fontsize_col = 10,
main = "Clinical Biomarker Heatmap by Patient Severity",

# Color palette
color = colorRampPalette(c("navy", "white", "firebrick3"))(100),
border_color = NA
)
}

```

This heatmap displays the biomarker values using manual z-score normalization

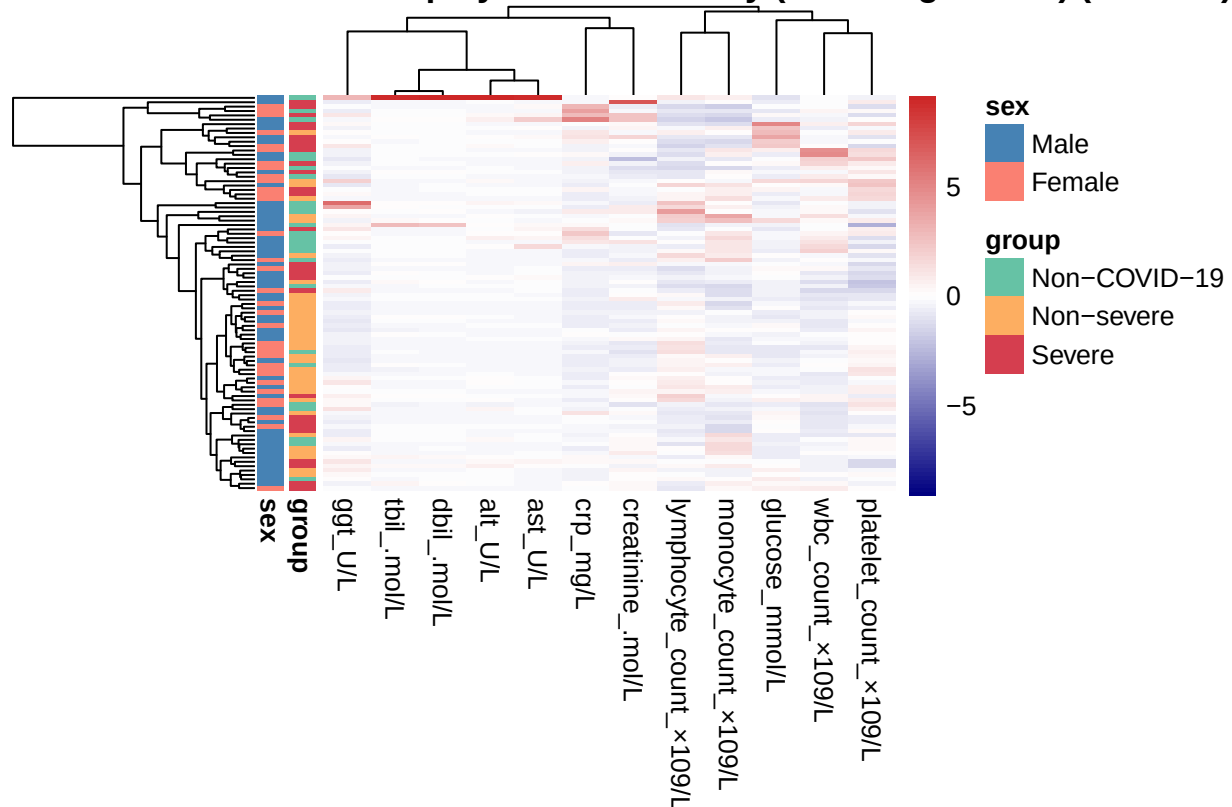
```

data_cleaned_no_JBDZ2 <- data_cleaned %>%
  filter(patient_id != "JBDZ2")

make_biomarker_heatmap(data_cleaned, FALSE, "(including JBDZ2)")

```

### Clinical Biomarker Heatmap by Patient Severity (including JBDZ2) (z-score)

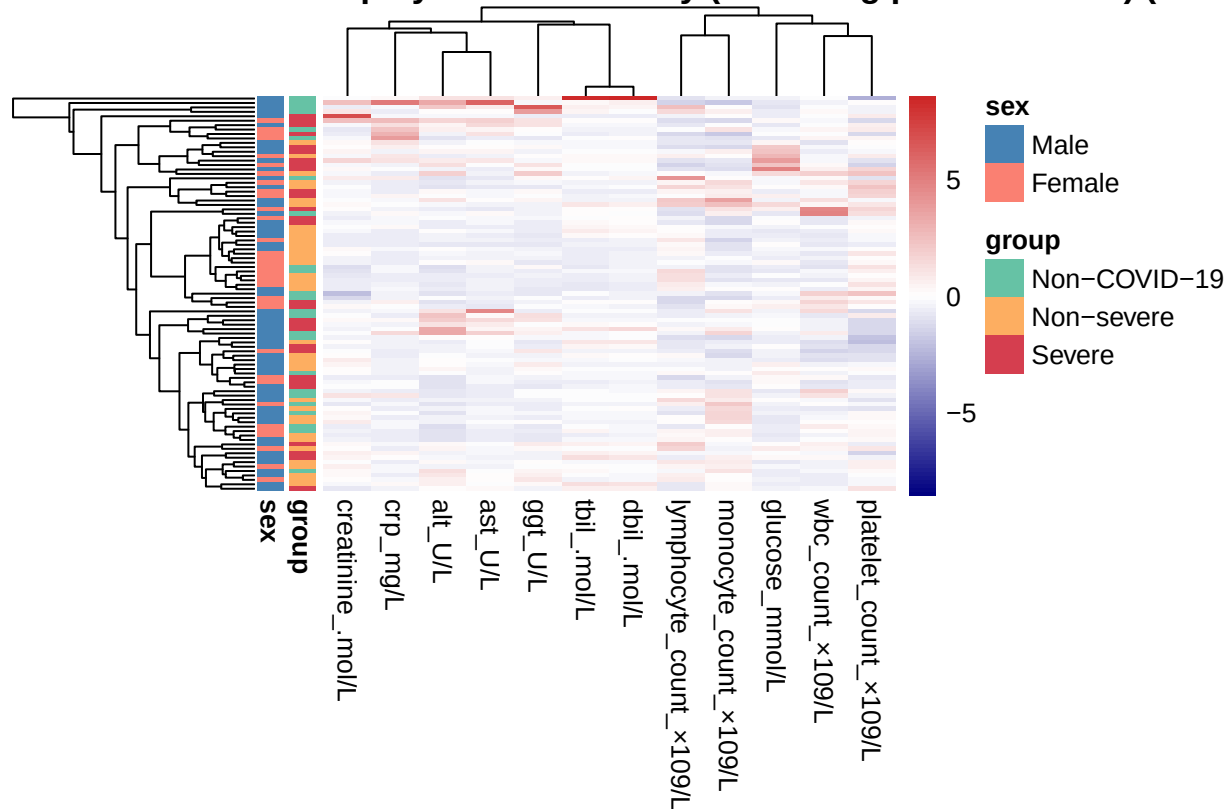


As previously explained, the non-COVID patient JBDZ2 presented extremely elevated liver enzymes, which could influence clustering after z-score normalization. So, we repeated the heatmap after excluding this patient.

This heatmap excludes the extreme patient JBDZ2 to evaluate the impact of this observation on clustering.

```
make_biomarker_heatmap(data_cleaned_no_JBDZ2, FALSE, "(excluding patient JBDZ2)")
```

### cal Biomarker Heatmap by Patient Severity (excluding patient JBDZ2) (z-sco

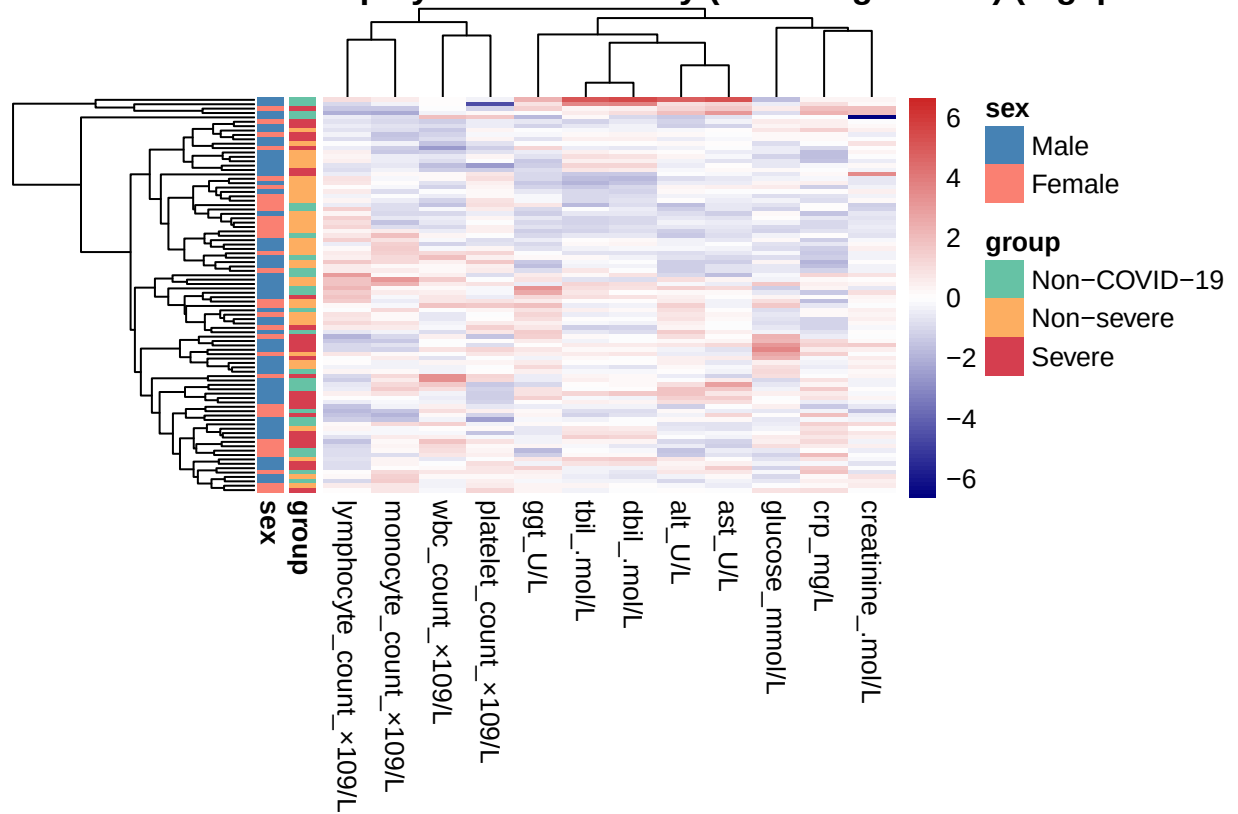


This heatmap incorporates a log-transformation prior to z-score normalization, reducing the influence of extreme values and skewed distributions. It allows assessment of whether transforming the data improves pattern detection and cluster structure.

```
make_biomarker_heatmap(data_cleaned, TRUE, "(including JBDZ2)")
```



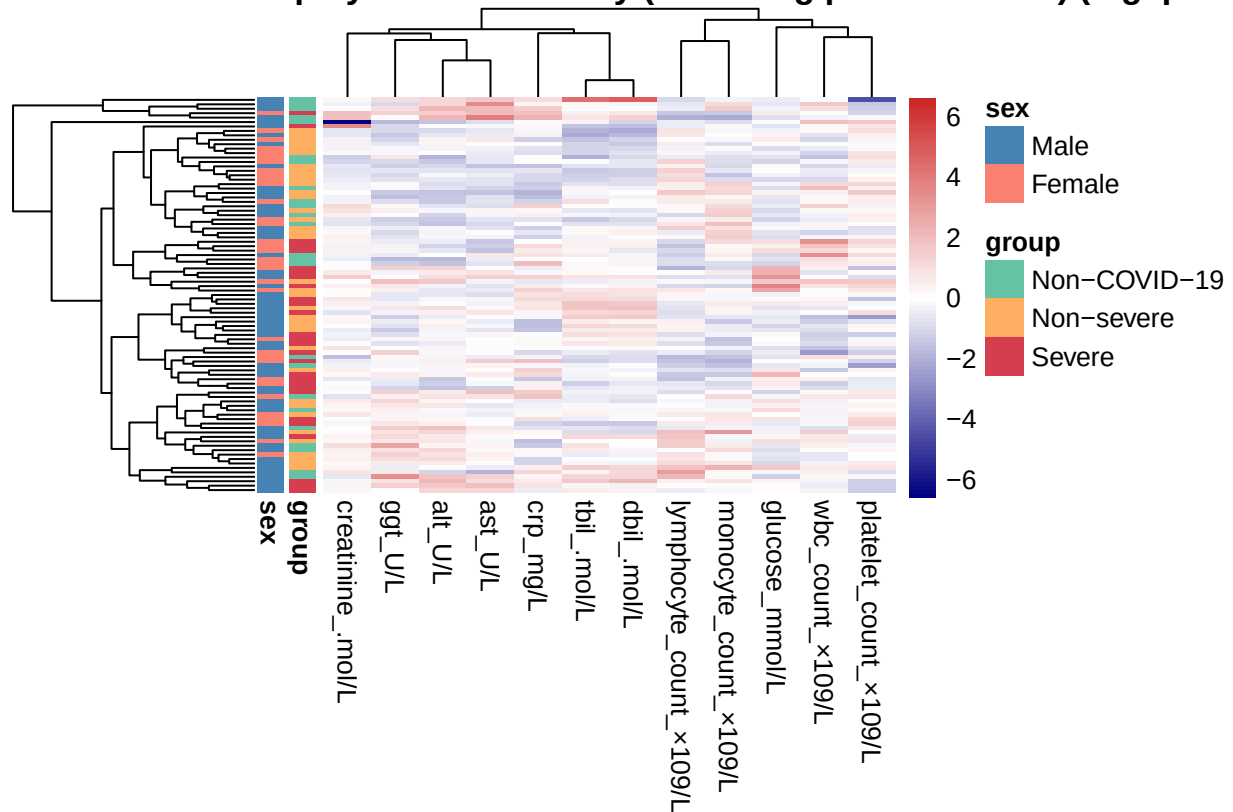
## cal Biomarker Heatmap by Patient Severity (including JBDZ2) (log1p + z-sco



This heatmap combines both the exclusion of the extreme patient and log-transformation.

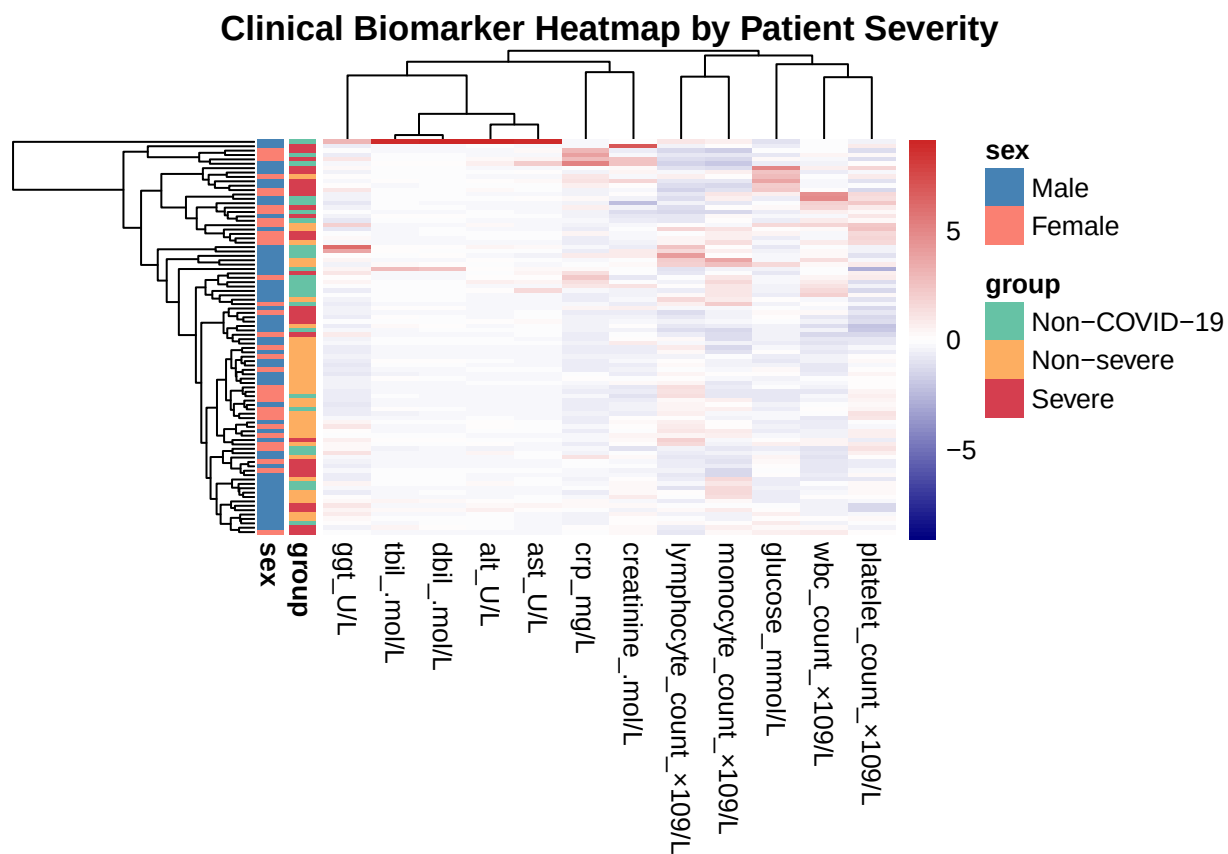
```
make_biomarker_heatmap(data_cleaned_no_JBDZ2, TRUE, "(excluding patient JBDZ2)")
```

### Biomarker Heatmap by Patient Severity (excluding patient JBDZ2) (log1p + z-



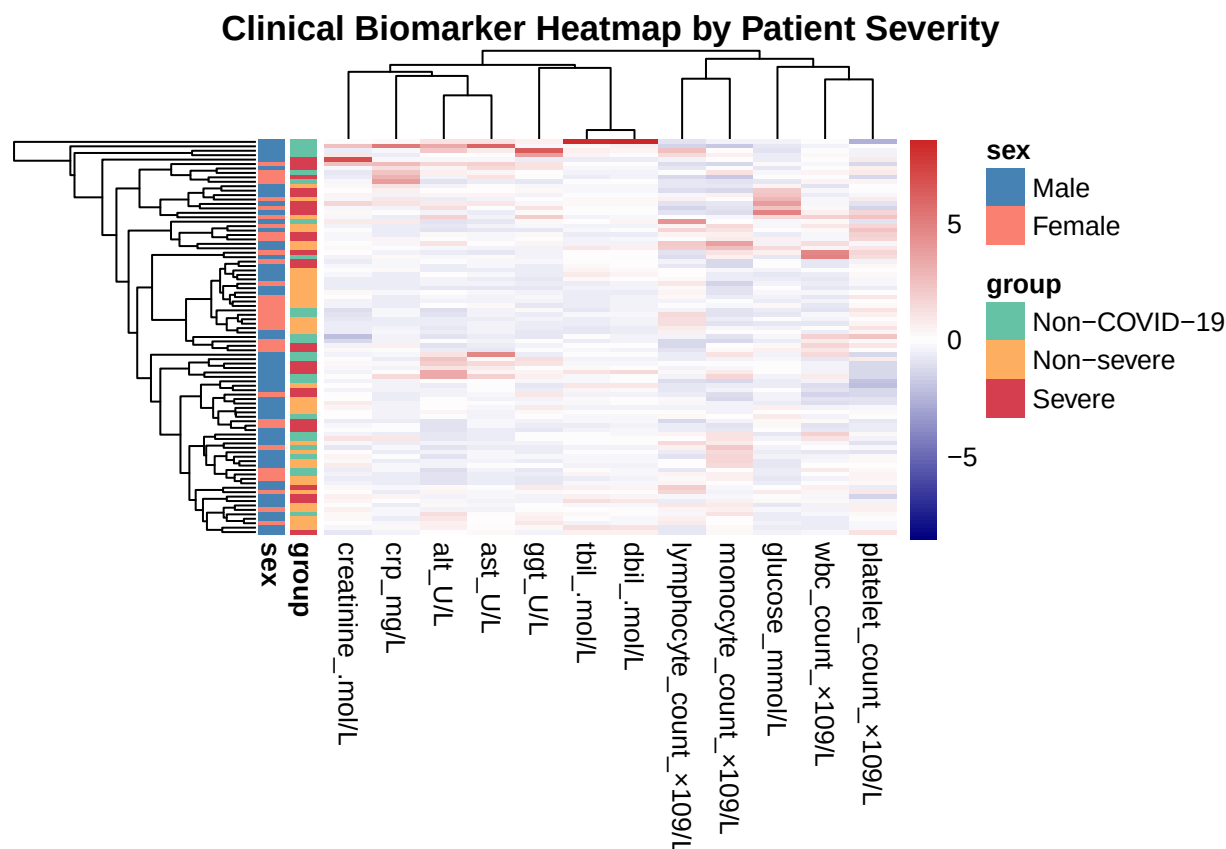
This heatmap is generated without prior manual scaling of the data. Instead, normalization is applied internally within the pheatmap function using column-wise scaling.

```
make_biomarker_heatmap_woPrevScale(data_cleaned, "(including JBDZ2)")
```



This heatmap is generated with internally normalization and excluding patient JBDZ2.

```
make_biomarker_heatmap_woPrevScale(data_cleaned_no_JBDZ2, "(excluding patient JBDZ2)")
```



Overall, the heatmaps do not reveal a clearly defined clustering pattern that separates patients by clinical group. But some tendencies can be observed, such as slightly higher biomarker levels in severe patients. This suggests that, while certain biomarkers are associated with disease severity they are not sufficient to form well-defined clusters.

Additionally, as we have checked different heatmaps for different preprocessing strategies we can say that the lack of clear clustering is not due to data quality issues or preprocessing choices, but rather reflects the biological complexity of the dataset.

## Conclusion

Write a concise conclusion (1 paragraphs) summarizing the main differences observed across the three patient groups based on the twelve clinical parameters. Support your statements with evidence from your analyses.

In conclusion, first, the analysis of the twelve clinical parameters reveals consistent differences across patient groups, with severe COVID-19 patients generally showing higher levels of inflammatory and liver-related biomarkers (such as CRP, ALT, AST, and bilirubin), while lymphocyte counts tend to be lower compared to non-severe and non-COVID-19 patients. These patterns are maintained across different visualization and preprocessing strategies, indicating that they reflect underlying biological differences. However, the absence of clear clustering in the heatmaps suggests that disease severity cannot be explained by a single or simple biomarker profile.

## session info

R version 4.5.2 (2025-10-31) Platform: x86\_64-pc-linux-gnu Running under: Ubuntu 22.04.5 LTS

Matrix products: default BLAS: /usr/lib/x86\_64-linux-gnu/blas/libblas.so.3.10.0 LAPACK: /usr/lib/x86\_64-linux-gnu/lapack/liblapack.so.3.10.0 LAPACK version 3.10.0

locale: [1] LC\_CTYPE=en\_GB.UTF-8 LC\_NUMERIC=C  
[3] LC\_TIME=es\_ES.UTF-8 LC\_COLLATE=en\_GB.UTF-8  
[5] LC\_MONETARY=es\_ES.UTF-8 LC\_MESSAGES=en\_GB.UTF-8  
[7] LC\_PAPER=es\_ES.UTF-8 LC\_NAME=C  
[9] LC\_ADDRESS=C LC\_TELEPHONE=C  
[11] LC\_MEASUREMENT=es\_ES.UTF-8 LC\_IDENTIFICATION=C

time zone: Europe/Madrid tzcode source: system (glibc)

attached base packages: [1] stats graphics grDevices utils datasets methods base

other attached packages: [1] pheatmap\_1.0.13 outliers\_0.15 patchwork\_1.3.2 cowplot\_1.2.0  
[5] applot\_0.2.9 ggplotify\_0.1.3 ggbreak\_0.1.7 ggpubr\_0.6.3  
[9] kableExtra\_1.4.0 knitr\_1.51 gt\_1.3.0 gtsummary\_2.5.0 [13] lubridate\_1.9.5 forcats\_1.0.1 stringr\_1.6.0  
dplyr\_1.2.0  
[17] purrr\_1.2.1 readr\_2.2.0 tidyr\_1.3.2 tibble\_3.3.1  
[21] ggplot2\_4.0.2 tidyverse\_2.0.0 readxl\_1.4.5

loaded via a namespace (and not attached): [1] gtable\_0.3.6 xfun\_0.57 rstatix\_0.7.3 tzdb\_0.5.0  
[5] yulab.utils\_0.2.4 vctrs\_0.7.2 tools\_4.5.2 generics\_0.1.4  
[9] pkgconfig\_2.0.3 RColorBrewer\_1.1-3 S7\_0.2.1 lifecycle\_1.0.5  
[13] compiler\_4.5.2 farver\_2.1.2 textshaping\_1.0.5 carData\_3.0-6  
[17] ggfun\_0.2.0 litedown\_0.9 htmltools\_0.5.9 sass\_0.4.10  
[21] yaml\_2.3.12 Formula\_1.2-5 pillar\_1.11.1 car\_3.1-5  
[25] abind\_1.4-8 commonmark\_2.0.0 tidyselect\_1.2.1 digest\_0.6.39  
[29] stringi\_1.8.7 labeling\_0.4.3 fastmap\_1.2.0 grid\_4.5.2  
[33] cli\_3.6.5 magrittr\_2.0.4 cards\_0.7.1 broom\_1.0.12  
[37] withr\_3.0.2 rappdirs\_0.3.4 scales\_1.4.0 backports\_1.5.0  
[41] timechange\_0.4.0 rmarkdown\_2.30 otel\_0.2.0 gridExtra\_2.3  
[45] ggsignif\_0.6.4 cellranger\_1.1.0 ragg\_1.5.2 hms\_1.1.4  
[49] evaluate\_1.0.5 viridisLite\_0.4.3 markdown\_2.0 gridGraphics\_0.5-1 [53] rlang\_1.1.7 glue\_1.8.0  
xml2\_1.5.2 svglite\_2.2.2  
[57] rstudioapi\_0.18.0 R6\_2.6.1 systemfonts\_1.3.2 fs\_2.0.1